RAPPORT FINAL PSC X24

Architecture Agentique et Hybride pour l'Extraction de Connaissances sur Code Propriétaire (Envision)

24 avril 2026

Dégot–Silvestre Gaétan
Dorchies Yoan
Kamdem Ivann
Thébault Guilhem
Guediche Adam

TABLE DES MATIÈRES

1 Introduction : Le Défi des DSLs Industriels **1**
1.1 Contexte : L'angle mort des Grands Modèles de Langage . . . 1
1.2 Objectifs . . . 1
1.3 Lokad et Envision : le terrain applicatif . . . 2
1.3.1 Envision : un DSL « 3-en-1 » pour la supply chain . . . 2
1.3.2 Une logique orientée batch, fichiers et décisions . . . 4
1.3.3 Du calcul à l'aide à la décision . . . 4

2 État de l'art : De la prédiction de mots à l'Agentification **5**
2.1 Émergence des LLM . . . 5
2.2 Hallucinations et leurs solutions . . . 5

3 Méthodologie et Architecture Implémentée **7**
3.1 Préparation des données : Du code brut aux "Chunks" sémantiques . . . 7
3.1.1 Le problème du découpage standard . . . 7
3.1.2 Notre solution : Le Envision Chunker . . . 7
3.1.3 La méthode RAPTOR . . . 7
3.2 Amélioration progressive de l'architecture grâce à LangGraph . . . 8
3.2.1 Évolution de la pipeline . . . 8
3.2.2 Fonctionnement de la boucle . . . 10
3.3 Outils . . . 10
3.3.1 Script Finder Tool et Gestion Centralisée des Chemins . . . 10
3.3.2 Tree tool . . . 11
3.3.3 RAG tool : l'outil de recherche sémantique . . . 11
3.3.4 Grep tool : la recherche lexicale exacte guidée par la structure du code . . . 13
3.3.5 Graph tool : un graphe de dépendances structurelles . . . 15
3.4 Query Transformers . . . 17
3.4.1 RAG Fusion . . . 17
3.4.2 HyDE . . . 17

4 Context Engineering **19**
4.1 Les instructions de base . . . 19
4.2 Le Distillation Tool . . . 19
4.3 Le Prior Evidence Tool . . . 20
4.4 Knowledge Bank / Accumulated Evidence / Execution History . . . 20
4.5 Motivations derrière cette implémentation . . . 20

5 Protocole d'Évaluation (Benchmarking) **22**
5.1 Le défi de la métrique : L'illusion de la similarité . . . 22
5.2 La solution polyvalente : "LLM-as-a-Judge" . . . 22
5.3 L'utilisation du dual cross-encoder . . . 22
5.4 La solution hybride . . . 23
5.5 Constitution du Benchmark . . . 23
5.6 Métriques de performance système . . . 23
5.7 Evaluation du benchmark . . . 24

6 Résultats **25**
6.1 Comparaison des Modèles de Langage 25
6.2 Analyse de l'Utilisation des Outils et de la Latence 26
6.3 Évaluation des Améliorations du RAG 27
6.4 Analyse détaillée des questions du Benchmark 28
6.5 Pistes d'amélioration 31

7 Gestion de Projet **32**
7.1 Déroulement et organisation du projet 32
7.2 Livrables et diffusion 33
7.3 Enseignements tirés 33
7.4 Conclusion 33

1
INTRODUCTION : LE DÉFI DES DSLS INDUSTRIELS

1.1 Contexte : L'angle mort des Grands Modèles de Langage

L'avènement des modèles de langage massivement pré-entraînés (LLM) a bouleversé les pratiques de développement logiciel. Sur des langages populaires comme Python ou Javascript, des outils comme GitHub Copilot agissent désormais comme des assistants quasi-omniscients. La raison de ce succès est simple : ces langages constituent une part significative du corpus d'entraînement public sur lequel ces modèles ont appris.

Cependant, la réalité industrielle est plus nuancée. De nombreuses entreprises, à l'instar de notre partenaire Lokad, fondent leur avantage compétitif sur des *Domain Specific Languages* (DSL) propriétaires. Dans le cas de Lokad, il s'agit d'Envision [1], un langage proche du SQL dédié à l'optimisation de la Supply Chain. Ici réside une première partie de notre problématique : les LLM standards (GPT-4, Mistral, Claude) n'ont quasiment jamais "lu" de code Envision.

La principale difficulté du projet repose ainsi sur la complexité réelle des architectures Envision mises à disposition : un compte typique est composé de plusieurs dizaines de scripts interconnectés, organisés en modules thématiques, et dont la documentation est partielle. Ces scripts réalisent des tâches variées allant du traitement de données à l'optimisation numérique.

1.2 Objectifs

Nous ne cherchons pas seulement à créer un moteur de recherche amélioré. Nous essayons de concevoir une **architecture agentique hybride**. Ce système doit être capable de décider de lui-même s'il doit effectuer une recherche sémantique (comprendre un concept) ou lexicale (trouver une variable exacte), et d'itérer sur sa propre réponse.

Nous détaillerons comment nous sommes passés d'une pipeline linéaire à l'utilisation de graphes d'états (via *LangGraph*), comment nous avons surmonté les limites du découpage de code (*chunking*), et comment nous avons mis en place un protocole d'évaluation automatisé (*LLM-as-a-Judge*) pour mesurer nos progrès de manière objective.

1.3 Lokad et Envision : le terrain applicatif

Notre partenaire industriel, Lokad, ne se présente pas comme un simple éditeur d'outils analytiques, mais comme une plateforme logicielle spécialisée dans l'optimisation quantitative de la supply chain [1]. Dans cette approche, les décisions opérationnelles ne sont pas paramétrées manuellement écran par écran : elles sont produites par des applications métiers spécialisées, écrites dans le DSL propriétaire Envision. Le site officiel met en avant une plateforme conçue pour couvrir des cas d'usage variés tels que la prévision, le réapprovisionnement, l'allocation ou encore la gestion opérationnelle de réseaux complexes.

1.3.1 • Envision : un DSL « 3-en-1 » pour la supply chain

Envision ne se résume pas à « un SQL propriétaire ». La documentation officielle le présente plutôt comme un langage spécialisé de la supply chain, pensé pour manipuler des tables, orchestrer des traitements et produire directement des restitutions [1]. On peut utilement le comprendre comme un compromis entre :

— un **langage déclaratif tabulaire**, proche de SQL par son travail sur les tables, les colonnes, les agrégations et les relations de données ;
— un **langage de processus contrôlé**, avec des mécanismes de flux et d'itération volontairement bornés, utilisés pour ordonner certains calculs ou scans séquentiels ;
— un **langage de restitution intégré**, capable de produire les tableaux, indicateurs et vues de pilotage directement depuis les scripts eux-mêmes.

Autrement dit, Envision peut être présenté comme un mélange de SQL, d'un sous-ensemble procédural proche d'un script Python très contraint, et de primitives de dashboarding intégrées. Ce n'est donc ni un simple langage de requêtes, ni un pur langage de programmation généraliste, ni un simple langage de visualisation.

Cette triple nature devient très visible dès qu'on lit quelques lignes réelles du corpus. On observe immédiatement : des `import` de modules, des `read` vers des fichiers de données, des `def process` exportés pour factoriser des calculs, des appels de fonctions sur des séries temporelles, puis des `write` et des `show table` qui matérialisent les sorties métier.

Pour préparer plus concrètement la lecture des graphes introduits ensuite, on peut regarder quelques motifs récurrents du corpus. Chacun fait apparaître un type de relation structurelle que notre représentation cherche précisément à expliciter.

— **Un script peut importer un module puis lire plusieurs fichiers d'entrée.** C'est la forme la plus immédiate de dépendance : *script* → *module importé* et *script* → *fichiers lus*.

```
import "/1. utilities/Modules/Global Parameters" as GP with
   DcUnit
   LinkLocationItemInspector
read "/Clean/Items.ion" as Items[Id unsafe] with
   "Sku" as Id : text
   Name : text
   SellPrice : number
read "{testStorage}Clean/Orders.ion" as Orders expect [Id, date] with
   "Sku" as Id : text
   "OrderDate" as date: date
   Quantity : number
```

— **Une logique métier peut être factorisée dans une fonction partagée.** Envision ne contient pas seulement des requêtes tabulaires : on y trouve aussi des calculs procéduraux bornés, exportés sous forme de fonctions. Le cas de `StockEvol` est typique.

```
export def process StockEvol(
   demand:number,
   desiredStockLvl: number,
   reorderCycle:number,
   leadtime:number,
   startStockonhand: number) with
   keep stockOnHand = 0
   keep stockOnOrder = 0
   ...
   return (stockOnHand, stockOnOrder, arrivingStockPreDay, newOrder)
```

— **Cette fonction peut ensuite être réimportée dans un script métier.** On voit alors apparaître une autre dépendance importante : *script → fonction définie ailleurs*.

```
import "/1. utilities/Modules/Functions" as Functions with
   StockEvol
History.MyStockOnHand,\
History.MyStockOnOrder,\
History.MyStockArriving,\
History.MyNewOrder =
StockEvol(History.WkQty,
   ItemsWeek.IdealStock,
   History.ReorderCycleWeeks,
   History.SupplierLTWeeks,
   Items.StartStock)
   by History.Id
   scan History.Mday
```

— **Le même langage sert enfin à produire des sorties et des vues métier.** Les scripts ne calculent donc pas seulement des valeurs : ils matérialisent aussi directement les résultats exploités par les utilisateurs métier.

```
write Categories as "Clean/PurchaseResults/EconomicParametersCategories.ion" with
   CategoryName = Categories.CategoryName
   AgressivenessFactor = Categories.AgressivenessFactorIni
write Scalar as "Clean/PurchaseResults/TargetsPurchaseList.ion" with
   FRTarget = Scalar.FRTarget
   budgetTarget = Scalar.budgetTarget
show table "Dispatch recommendations" with
   Skus.Sku
   Skus.Ref
   Skus.DispatchQuantity as "Qty to Dispatch"
```

Ces extraits suffisent déjà à faire émerger l'intuition qui sera centrale pour la suite du rapport : un script Envision n'est pas isolé. Il peut importer un module, réutiliser une fonction définie ailleurs, lire plusieurs fichiers d'entrée, produire des artefacts de sortie, puis exposer des résultats dans des vues métier. C'est précisément cette structure d'interdépendances que nous avons exploité dans nos divers outils.

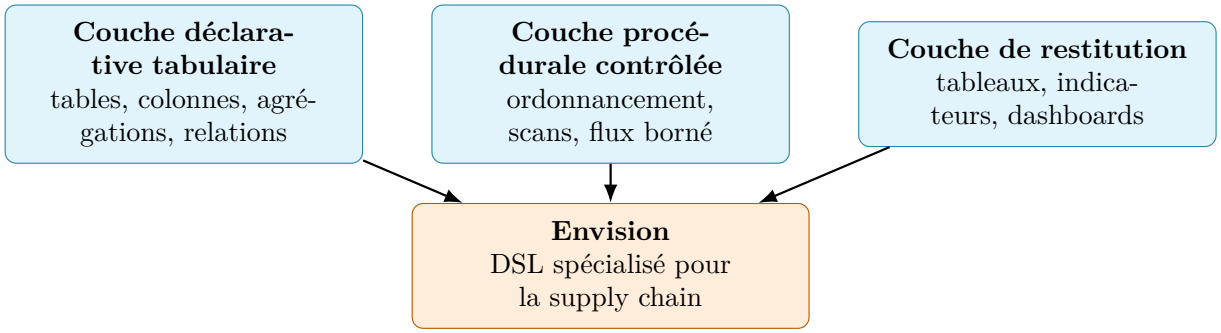

Figure 1 – Lecture synthétique d'Envision : un DSL qui combine manipulation tabulaire, processus de calcul et restitution métier dans un même langage.

1.3.2 • Une logique orientée batch, fichiers et décisions

La documentation officielle insiste aussi sur le caractère très concret et orienté exécution d'Envision : les scripts lisent des fichiers d'entrée, produisent des tables intermédiaires, sauvegardent des résultats et alimentent des vues de pilotage [1]. Dans le corpus que nous avons étudié chez Lokad, cette logique se matérialise par deux arborescences fortement structurées :

— une arborescence **scripts**, qui contient les traitements Envision ;
— une arborescence **data**, qui contient les données d'entrée, les jeux nettoyés et certains artefacts manuels.

Le point important, pour notre PSC, est que l'ordre de lecture du projet devient en pratique presque visible dans les noms. Dans notre dépôt, les dossiers et fichiers préfixés par `1.`, `2.`, `3.`, etc. rendent immédiatement lisible une convention de progression : utilitaires et préparation des données, contrôles de qualité, inspecteurs analytiques, optimisation, planification long terme, documentation. Cette convention n'est pas une propriété intrinsèque du langage Envision ; c'est une propriété organisationnelle extrêmement utile du corpus réel que nous devons naviguer.

1.3.3 • Du calcul à l'aide à la décision

Cette organisation n'est pas purement académique : elle sert à produire des décisions opérationnelles concrètes. Le site officiel Lokad met en avant des cas d'usage orientés prévision, réapprovisionnement, allocation, tarification et plus généralement pilotage quantitatif de la supply chain. Dans des secteurs à forte contrainte, cela signifie que les scripts Envision ne servent pas seulement à calculer des indicateurs, mais bien à préparer des décisions qui ont un impact direct sur l'approvisionnement, le stock ou la disponibilité opérationnelle.

Dans un tel contexte, un projet client Envision ressemble moins à un « petit script » qu'à une application batch métier complète : ingestion de données, nettoyage, calculs tabulaires, optimisation, génération de tableaux de bord et restitution d'actions recommandées. Cette richesse fait la force de la plateforme, mais elle crée aussi la difficulté qui motive notre PSC : pour un humain comme pour un LLM, il devient vite coûteux de retrouver *où* une métrique est calculée, *quel* script lit un fichier donné, *où* une suggestion d'approvisionnement est produite, ou *quel* tableau de bord expose un résultat.

Cette difficulté de navigation justifie directement notre objectif : construire un assistant capable de retrouver, relier et expliquer les éléments pertinents d'un projet Envision réel, sans donner au modèle un accès brut et illimité à l'ensemble du dépôt ni aux données confidentielles du client.

2
ÉTAT DE L'ART : DE LA PRÉDICTION DE MOTS À L'AGENTIFICATION

2.1 Émergence des LLM

Les premières idées de réseaux de neurones datent des années 40, mais ceux-ci ne connaissent pas un franc succès. À partir des années 80, le développement de l'algorithme de rétropropagation leur permet de revenir sur le devant de la scène mais toujours sans grandes réussites car les modèles ont alors une taille intermédiaire qui limite les possibilités de cette technologie. Ce n'est qu'au début des années 2010 que les performances des réseaux de neurones ne cessent d'augmenter grâce à l'amélioration des capacités de calcul des GPU qui permet l'émergence des réseaux de neurones profonds et la mise en place de nouvelles méthodes.

Si des méthodes de Natural Language Processing (NLP) à l'aide de réseaux de neurones sont étudiées depuis longtemps, le traitement et la génération de texte ont connu des progrès phénoménaux depuis 2017 avec l'implémentation du mécanisme d'attention et des transformers, dans l'article *Attention is all you need* [2], menant à la sortie de ChatGPT 3 en 2022. Ces progrès se sont poursuivis avec l'apparition de modèles de raisonnements, toujours plus efficaces notamment pour la génération de code.

Afin d'améliorer les performances des LLM, la tendance actuelle semble être à l'"agentification". Une IA agentique améliore itérativement la réponse d'un LLM en lui donnant accès à une grande variété d'outils (ex : une calculatrice, un terminal pour tester le code avant de le fournir à l'utilisateur, la recherche internet, etc) et en lui permettant d'adapter progressivement sa réponse grâce aux résultats obtenus à partir des outils. La sortie de ChatGPT-5 est un grand pas dans cette direction car c'est un agent IA qui rassemble en un seul modèle tous les modèles de la génération précédente d'OpenAI en choisissant de façon autonome lequel est le plus adapté à la requête de l'utilisateur.

2.2 Hallucinations et leurs solutions

Cependant, les résultats des LLM sont parfois limités : les LLM étant entraînés pour donner une réponse quoi qu'il arrive, ils ont tendance à proposer une réponse fausse avec certitude face à des problèmes difficiles, ce que l'on appelle des hallucinations [3]. C'est d'autant plus le cas pour des DSL, que les LLM connaissent mal, et si la seule documentation du langage suffit à leur faire produire du code syntaxiquement correct, celui-ci donne rarement le bon résultat.

Une première solution simple à implémenter pour éviter les hallucinations est de formuler la requête donnée au LLM avec précision et ingéniosité. Il a par exemple été remarqué qu'on obtient de meilleurs résultats en moyenne si on précise au modèle de langage son rôle ; c'est ce qu'on appelle le *Persona Pattern*. Ainsi la requête : "Tu es mon professeur de mathématiques, explique moi et démontre moi l'inégalité de Cauchy-Schwarz" donnera probablement un meilleur résultat que la simple requête : "Explique-moi et démontre-moi l'inégalité de Cauchy-Schwarz". Cette astuce est une des nombreuses techniques qui constituent le *Prompt Engineering* [4].

Avec l'émergence des agents IA évoquée plus haut, une nouvelle couche de complexité s'est ajoutée au Prompt Engineering, on parle alors de *Context Engineering*. Cette discipline ne se contente plus d'optimiser la formulation des instructions, mais se focalise sur la sélection et l'organisation stratégique des données fournies au modèle. L'enjeu principal réside dans la gestion du rapport signal-sur-bruit : un excès d'informations peut "distraire" l'agent et dégrader ses performances, tandis qu'un contexte trop restreint limite sa capacité d'action.

Le Context Engineering repose ainsi sur une structuration rigoureuse (notamment via l'usage du format JSON ou encore de balises XML) pour distinguer les outils, les documents et les consignes, ainsi que sur des mécanismes de filtrage dynamique. En optimisant ce flux informationnel, on permet à l'agent de naviguer efficacement dans des workflows itératifs complexes, tout en maîtrisant la latence et les coûts grâce à des techniques comme la mise en cache du contexte [5].

Une approche alternative, plus fondamentale mais néanmoins digne d'intérêt, consisterait à utiliser des outils de recherche lexicale, dont la commande grep est l'exemple le plus emblématique. Cette méthode se base sur la recherche de correspondances exactes de chaînes de caractères ou d'expressions régulières directement dans les fichiers de code source. Si cette technique est dépourvue de toute compréhension du sens — elle ne ferait pas le lien entre "calculer le stock" et une variable nommée "inventory-level" — elle offre une rapidité et une précision inégalées pour des requêtes littérales, comme retrouver toutes les occurrences d'un nom de fonction ou d'une variable spécifique.

Une autre solution possible à ce problème est d'utiliser des méthodes de *Retrieval Augmented Generation* (RAG) [6], qui consistent à retrouver les éléments de documentation liés à la requête de l'utilisateur et les fournir directement au LLM pour améliorer ses réponses. Pour cela, on peut par exemple utiliser l'indexation sémantique avancée, en représentant le texte sous forme de vecteurs (appelés *embeddings*) tels que deux textes sémantiquement similaires soient représentés par des vecteurs proches [7]. Des modules comme *Sentence Transformers* permettent de générer ces vecteurs, et d'autres tels que *FAISS* (Facebook AI Similarity Search) servent à formuler des recherches de similarité sur des vecteurs de grande dimension via des calculs de proximité selon différentes métriques.

Cependant, la recherche sémantique classique par similarité cosinus présente des limites face à des requêtes asymétriques ou du vocabulaire très spécifique. Pour pallier ce fossé sémantique, des techniques d'augmentation de requêtes telles que HyDE (Hypothetical Document Embeddings) ont vu le jour [8]. Cette méthode exploite un LLM pour générer une réponse hypothétique à la question de l'utilisateur, et c'est le vecteur de cette réponse générée qui est ensuite utilisé pour chercher les documents pertinents dans la base. Le modèle d'encodage agit alors comme un filtre qui élimine les informations factuellement fausses générées dans l'hypothèse, tout en capturant les motifs sémantiques pertinents.

Dans le contexte de bases de code complexes, il est souvent nécessaire de comprendre la structure globale d'un projet plutôt que de lire de simples fragments de code isolés. L'architecture RAPTOR (Recursive Abstractive Processing for Tree-Organized Retrieval) répond à ce besoin en regroupant et en résumant récursivement les fragments de texte pour construire un arbre hiérarchique de connaissances [9]. Lors de la recherche, le système peut ainsi extraire des informations à différents niveaux d'abstraction, ce qui améliore significativement les performances sur des questions nécessitant une compréhension holistique et un raisonnement multi-étapes.

Enfin, pour maximiser la précision contextuelle, la simple récupération de documents initiaux est aujourd'hui supplantée par des architectures intégrant un reclassement (reranking) neuronal. L'utilisation de modèles d'attention croisée (Cross-Encoder), popularisée par les travaux de Nogueira et Cho [10], permet d'évaluer la pertinence en traitant simultanément la question et le document candidat au sein du même réseau. Bien que plus coûteuse en calcul, cette méthode capture des relations sémantiques fines inaccessibles aux simples recherches vectorielles et s'avère indispensable pour hisser les extraits de code les plus exacts en tête de liste.

Peu importe la technique utilisée, les données ainsi extraites devant être fournies directement dans la requête, la formulation précise de cette dernière est un aspect central du RAG [11]. Le RAG utilise donc plusieurs techniques de Prompt Engineering.

Une dernière possibilité est le fine-tuning de modèles pré-entraînés : plutôt que de réentraîner un modèle spécialisé à partir de rien, ce qui est très coûteux, il est possible de spécialiser l'entraînement d'un modèle généraliste pour l'adapter à un cas d'usage spécifique [12]. Cependant, pour mettre en place cette solution efficacement, il faut une grande quantité d'exemples de questions/réponses. Cela peut donc conduire à l'utilisation d'un autre LLM (souvent plus gros) afin de générer d'autres exemples [13]. Pour ces raisons, par manque de temps et afin de préserver la confidentialité des données de Lokad, nous n'avons pas implémenté de fine tuning dans notre PSC.

3
MÉTHODOLOGIE ET ARCHITECTURE IMPLÉMENTÉE

Notre approche ne s'est pas construite en une seule étape. Elle est le fruit d'itérations successives, partant d'un constat d'échec sur les méthodes classiques pour aboutir à une architecture hybride et agentique.

Afin de fluidifier l'organisation, nous avons rapidement choisi d'utiliser une structure très modulaire dans laquelle des classes abstraites définissent les interfaces de chaque étape de la pipeline. Chaque nouvel outil ou amélioration est alors développé selon ces interfaces et peut donc être permuté avec un autre sans créer de conflits. Cela nous a permis de travailler de manière indépendante sur différentes étapes du processus, et de facilement comparer différentes solutions. Par exemple, nous avons implémenté une classe pour chaque LLM utilisé (Mistral, Llama...) héritant d'une même classe abstraite, que l'on pouvait donc utiliser dans le reste du projet sans se soucier des différences entre les API de chaque modèle. Cette manière de procéder se retrouve dans tout le reste de la pipeline.

3.1 Préparation des données : Du code brut aux "Chunks" sémantiques

La qualité d'un système RAG dépend avant tout de la qualité de ses données. Contrairement à du texte littéraire, le code source d'Envision possède une structure hiérarchique forte qui ne supporte pas un découpage arbitraire.

3.1.1 • Le problème du découpage standard

Les méthodes classiques de *chunking* (découpage par nombre fixe de tokens ou par séparateurs de paragraphes) s'avèrent destructrices pour du code.

- Si un découpage coupe une fonction en deux, le modèle perd la signature de la fonction (ses arguments) ou son corps logique.
- Le contexte est souvent situé en début de fichier (déclaration des tables), information cruciale pour comprendre les lignes situées 500 lignes plus bas.

3.1.2 • Notre solution : Le Envision Chunker

Nous avons développé un module de parsing spécifique (`EnvisionParser`) qui analyse la structure syntaxique des fichiers `.nvn`. Plutôt que de découper aveuglément, nous identifions des blocs logiques :

- **Blocs de définition :** `table`, `expect`, `read`.
- **Blocs logiques :** Délimitations par les commentaires spéciaux `///` utilisés dans Envision.

Le `EnvisionChunker` regroupe ensuite ces segments en unités cohérentes, tout en veillant à ce que chaque *chunk* conserve les métadonnées essentielles (nom du fichier source, bibliothèques importées) pour garantir sa compréhension isolée par le LLM. Si un bloc dépasse le seuil maximal de tokens autorisé, le `chunker` procède à sa division. Pour pallier la perte de contexte aux points de rupture, nous avons implémenté une stratégie de chevauchement de quelques lignes entre les segments adjacents, facilitant l'analyse du code situé à la frontière entre deux sous-chunks.

3.1.3 • La méthode RAPTOR

Pour améliorer les résultats du RAG sur des questions générales, sans lien direct avec un morceau de code spécifique, nous avons implémenté la méthode RAPTOR (Retrieval-Augmented Generation with Tree-based Retrieval). Le processus commence par l'analyse des fichiers source au niveau de base, où les documents sont parsés, découpés en chunks et enrichis de résumés pré-calculés, avant d'être transformés en embeddings

vectoriels pour faciliter les comparaisons de similarité. Ensuite, une construction récursive de l'arbre s'opère : les chunks sont regroupés en clusters à l'aide d'algorithmes de réduction de dimension (UMAP) et de modélisation probabiliste (GMM), puis chaque cluster fait l'objet d'un résumé consolidé, créant ainsi des niveaux hiérarchiques successifs jusqu'à atteindre une vue d'ensemble.

Lors du RAG, les résumés des clusters peuvent être renvoyés au même titre que les chunks issus des scripts d'origine, l'idée étant que ces résumés sont plus pertinents pour des questions générales. Les chemins des fichiers dont viennent les chunks du cluster sont également fournis au LLM, lui permettant de les explorer plus en détail lors d'une itération ultérieure.

3.2 Amélioration progressive de l'architecture grâce à LangGraph

Plutôt qu'une pipeline linéaire (Question → Recherche → Réponse), nous avons implémenté une architecture cyclique basée sur des graphes d'états (`StateGraph`), utilisant la librairie **LangGraph** [14].

3.2.1 • Évolution de la pipeline

La pipeline prévisionnelle que nous avons établie dans la proposition détaillé était la suivante :

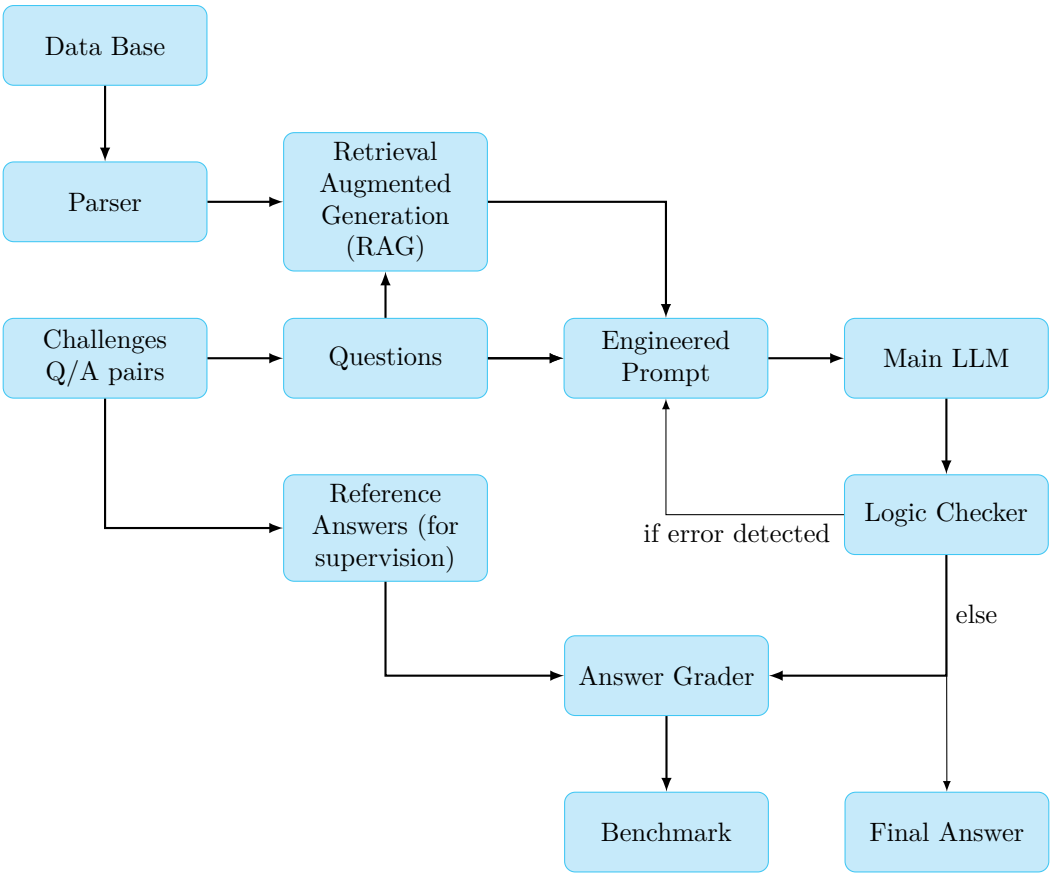

Figure 2 – Pipeline prévisionnelle issue de la proposition détaillée

Dans un premier temps, nous l'avons implémentée sans coder de boucle rétroactive mais nous nous sommes

rendu compte que le RAG était peu adapté à certains types de questions comme les questions de comptage. Nous avons donc remplacé le bloc RAG entre Questions et Engineered Prompt par la sous-pipeline suivante :

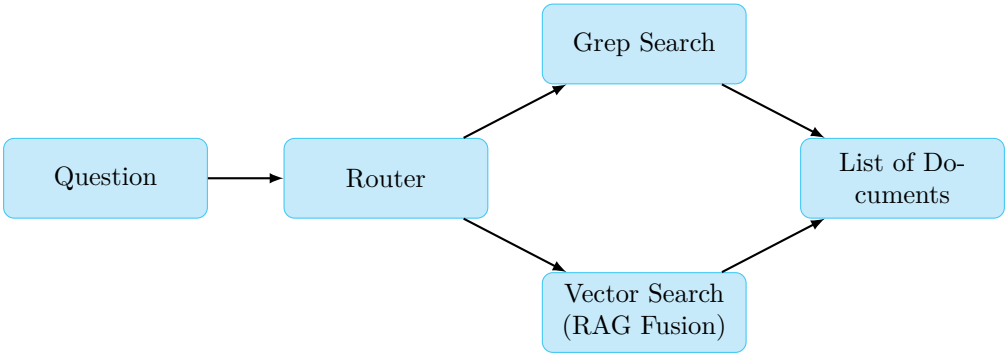

Figure 3 – Workflow avec routeur qui remplaçait le bloc RAG dans la pipeline prévisionnelle

Cette nouvelle architecture était plus performante mais manquait d'une boucle rétroactive pour les questions complexes. Or plusieurs études récentes démontrent l'intérêt de passer d'un pipeline RAG statique à un système d'agents autonomes permettant plusieurs itérations de réflexion [15] [16]. Nous avons donc repensé la pipeline prévisionnelle pour que cette boucle soit vraiment le cœur du modèle et avons abouti à ce workflow agentique :

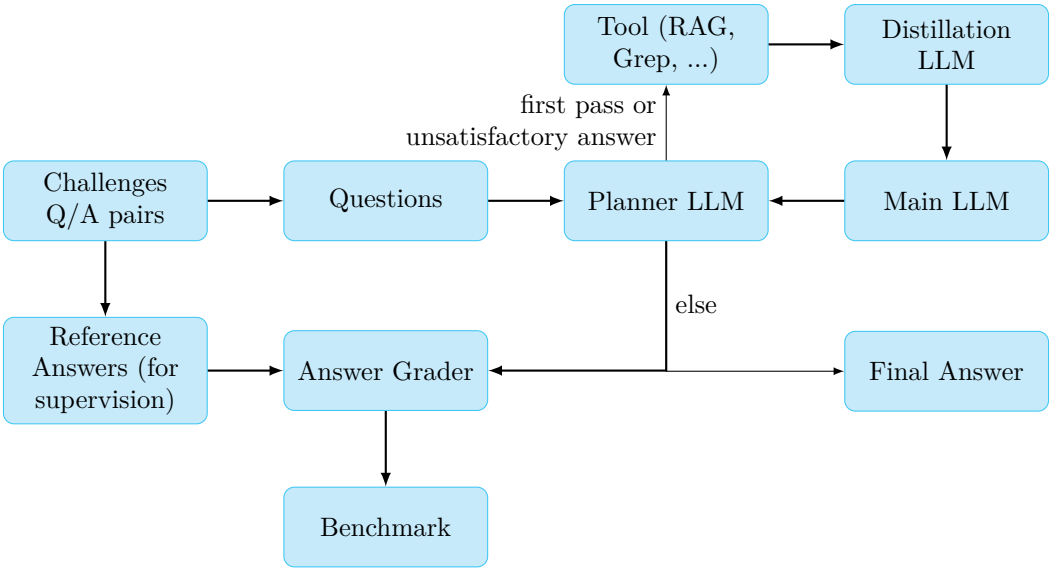

Figure 4 – Pipeline Agentique qui correspond à l'état actuel de notre architecture

À ce jour, cette architecture reste au cœur de notre système, à une optimisation près. Afin d'éviter de solliciter inutilement le `Main LLM` lorsqu'un outil ne renvoie aucun résultat (en raison d'arguments invalides ou d'une recherche infructueuse), nous redirigeons immédiatement le retour d'erreur vers le `Planner LLM`. Cette boucle de correction rapide, appelée *follow-up call* dans notre base de code, oblige le planificateur à ajuster ses paramètres sans répéter son erreur initiale. L'avantage technique de cette approche réside dans l'exploitation du *KV-caching* : le modèle conservant en mémoire les matrices d'attention déjà calculées pour une séquence initiale de tokens identique, cette itération s'appuie directement sur le contexte de la génération précédente. Ainsi, le coût de calcul additionnel est minime, puisque l'attention n'est recalculée que pour le nouveau segment de prompt ajouté à la fin de l'historique.

3.2.2 • Fonctionnement de la boucle

Le cycle se déroule comme suit :

1. **Planification :** L'agent analyse la demande. S'il lui manque des informations, il décide d'appeler un outil (`Tool Call`).
2. **Exécution :** L'outil (RAG ou Grep) s'exécute et retourne des données brutes.
3. **Observation et Réflexion :** L'agent reçoit ces données. Il peut décider qu'elles sont suffisantes et générer la réponse, ou qu'elles sont incomplètes et formuler une nouvelle requête (ex : "J'ai trouvé la table, maintenant je cherche la colonne").

Cette capacité de rétroaction et d'itération est ce qui distingue fondamentalement notre prototype d'un simple moteur de recherche : il mime le comportement d'un développeur humain qui navigue dans le code.

3.3 Outils

Pour permettre au modèle d'appeler les outils que l'on a définis, nous utilisons la syntaxe dite "OpenAI-compatible", introduite par OpenAI et prise en charge par les APIs de la plupart des grands modèles. Lorsque l'on veut que le modèle appelle un tool, nous lui fournissons la liste des tools au format suivant :

```
{
    "type": "function",
    "function": {
        "name": "tool_name",
        "description": "Ce que fait le tool",
        "parameters": {
            "type": "object",
            "properties": { /* arguments */ },
            "required": [ /* arguments obligatoires */ ]
        }
    }
}
```

Cette syntaxe force (ou, pour certains modèles, encourage fortement mais sans garantie) le choix de l'un des outils listés. Le modèle répond dans un format standardisé indiquant notamment l'outil et les arguments choisis.

3.3.1 • Script Finder Tool et Gestion Centralisée des Chemins

Afin de permettre à l'agent de consulter l'intégralité d'un code source lorsqu'une vue globale est nécessaire, un premier outil que nous avons implémenté est le `script_finder_tool`. Ce développement a également été l'occasion de concevoir une gestion centralisée de l'arborescence des fichiers utilisée par tous les outils.

En pratique, les scripts extraits des comptes clients se présentent souvent sous la forme d'un *dump* de fichiers plats nommés par des identifiants bruts (par exemple `67984.nvn`). Pour que l'agent puisse raisonner sur une structure de projet cohérente, notre système reconstruit une arborescence virtuelle complète à l'aide d'un simple fichier de correspondance (`mapping.txt`). Cette architecture rend notre solution extrêmement modulaire : pour analyser un nouveau compte client, il suffit de fournir le dossier contenant les scripts en vrac et son fichier de *mapping* associé. Le système gère alors automatiquement et de manière transparente la traduction entre les identifiants techniques et les chemins métiers compréhensibles par le LLM.

Le `script_finder_tool` s'appuie sur cette mécanique pour retrouver et lire n'importe quel fichier à partir d'un fragment de son chemin d'origine. Cependant, dans sa version actuelle, cet outil renvoie systématiquement l'intégralité du texte du script demandé. En raison de la saturation rapide de la fenêtre de contexte que cela engendre, nous avons dû brider son utilisation via les directives système (*« Use RARELY and only when*

necessary due to high token cost »), forçant ainsi le `Planner LLM` à privilégier des outils plus ciblés comme le RAG ou le grep.

Une piste d'amélioration solide consisterait à faire évoluer cet outil pour qu'il accepte des arguments de plage de lecture (par exemple `start_line` et `end_line`). Cela permettrait à l'agent d'inspecter chirurgicalement le code situé juste au-dessus ou en dessous d'un *chunk* renvoyé par le RAG, sans inonder sa mémoire de travail. Si l'implémentation Python de cette fonctionnalité est triviale, nous avons sciemment choisi de ne pas l'intégrer au cours des dernières semaines du PSC. En effet, dans un flux de travail agentique, atteindre un point d'équilibre où le modèle choisit judicieusement entre plusieurs outils concurrents requiert un *prompt engineering* millimétré. Modifier l'utilisation du `script_finder_tool` risquait de perturber celle du RAG et donc de déstabiliser cet équilibre précaire que nous avions atteint.

Néanmoins, l'intégration de cette pagination ciblée reste une évolution naturelle et nécessaire si l'on souhaite se rapprocher des standards actuels du marché des agents codeurs autonomes (comme GitHub Copilot ou Cursor).

3.3.2 • Tree tool

Pour fournir l'arborescence de certains fichiers sources au modèle, nous lui donnons un outil lui permettant de recevoir la représentation de l'arborescence au format usuel employé par exemple par l'outil tree de Linux, et donc plus facilement compris par le modèle.

En pratique, cet outil est rarement appelé explicitement par le planificateur car nous donnons une version condensée de l'arborescence de la base de code au début de chaque prompt.

3.3.3 • RAG tool : l'outil de recherche sémantique

L'implémentation initiale de notre système RAG reposait sur FAISS (Facebook AI Similarity Search), une solution efficace pour le prototypage rapide, mais fondamentalement limitée par son approche unidimensionnelle. Dans cette première version, la recherche s'effectuait exclusivement via des vecteurs denses (*embeddings*) qui capturaient le sens sémantique global des requêtes, mais peinaient face au vocabulaire très spécifique d'une base de code (noms de variables exacts, UUIDs, ou acronymes métiers de Lokad/Envision). Pour pallier cette faiblesse et permettre à l'agent de cibler des fichiers spécifiques ou des mots-clés, nous avions mis en place un système de reclassement manuel codé en dur en Python. Après avoir récupéré les K meilleurs résultats de FAISS, le script appliquait des multiplicateurs mathématiques artificiels : le score d'un chunk était augmenté de quelques pourcents s'il contenait un mot-clé demandé, ou pénalisé s'il ne correspondait pas à l'expression régulière (*regex*) du chemin de fichier spécifié. Bien que fonctionnelle sur une petite échelle, cette méthode était algorithmiquement fragile. Elle obligeait le système à manipuler des scores post-recherche, ce qui signifiait que si FAISS ne remontait pas le bon fichier dans son top 30 initial, le système de pénalités Python ne pouvait tout simplement pas corriger le tir, la bonne réponse ayant déjà été éliminée de la sélection.

Pour franchir un cap en termes de précision, nous avons migré la base vectorielle vers Qdrant, ce qui nous a permis de débloquer la puissance de la recherche hybride en combinant une base de données "Dense" (comme FAISS) et une base de données "Sparse" qui permet de faire des recherches par fréquence de tokens. Pour cette base de données sparse, nous utilisons BM25 (Best Matching 25) [17] qui est une fonction de classement fréquentielle estimant la pertinence d'un document D par rapport à une requête Q. Le score de pertinence est calculé selon la formule suivante :

$$\text{Score}(D, Q) = \sum_{q \in Q} \text{IDF}(q) \cdot \frac{f(q, D) \cdot (k_1 + 1)}{f(q, D) + k_1 \cdot (1 - b + b \cdot \frac{|D|}{\text{avgdl}})} \tag{1}$$

où $q \in Q$ sont les tokens compris dans Q. Pour bien appréhender la mécanique de cette formule, il convient de la décomposer en trois axes fondamentaux qui agissent de concert :

— **Le poids intrinsèque du terme (La composante IDF) :** La partie $\mathrm{IDF}(q)$ agit comme un filtre de rareté. Elle est classiquement calculée selon la formule suivante :

$$\mathrm{IDF}(q) = \ln\left(\frac{N - n(q) + 0.5}{n(q) + 0.5} + 1\right)$$

où N représente le nombre total de documents dans la collection (la base de code), et $n(q)$ est le nombre de documents contenant le terme q. Elle part du principe mathématique qu'un mot très commun dans la base de code (où $n(q)$ s'approche de N) verra son score s'effondrer vers zéro, car il apporte peu d'information discriminante. À l'inverse, un terme technique rare (où $n(q)$ est très petit) verra son score multiplié de façon significative.

— **La saturation de la fréquence (Le paramètre $k_1 \in [1.2, 2.0]$) :** La fraction centrale de l'équation gère la fréquence d'apparition du terme $f(q, D)$, c'est-à-dire le nombre de fois où le mot est trouvé dans le document. Sans le paramètre constant k_1, un document répétant 100 fois le même mot écraserait tous les autres résultats. Ce paramètre k_1 impose une courbe asymptotique : la première apparition d'un mot donne un score fort, la deuxième un peu moins, jusqu'à plafonner mathématiquement vers $(k_1 + 1)$. Cela traduit l'idée sémantique que "mentionner un sujet 5 fois prouve qu'on en parle, le mentionner 50 fois n'apporte plus de preuve supplémentaire".

— **La normalisation par la longueur (Le paramètre $b \approx 0.75$) :** Le dénominateur intègre le ratio $\frac{|D|}{\mathrm{avgdl}}$, qui compare la longueur du document évalué $|D|$ à la longueur moyenne des documents de la base `avgdl`. Par pure probabilité, un fichier de 3000 lignes a beaucoup plus de chances de contenir accidentellement les mots de la requête qu'un petit fichier de 50 lignes. Le paramètre constant b (généralement fixé à 0.75, mais obligatoirement compris entre 0 et 1) vient pondérer cette injustice statistique en pénalisant les documents excessivement longs, assurant ainsi que la concision et la densité de l'information sont récompensées.

Avec Qdrant, nous avons mis en place une architecture sophistiquée à 4 flux de recherche simultanés (2 flux *denses* et 2 flux *sparses*) qui se balancent mutuellement grâce au Reciprocal Rank Fusion (RRF) [18], qui favorise les documents récupérés dans plusieurs flux différents :

$$RRF(d) = \sum_{r \in R} \frac{1}{k + r(d)}$$

où chaque élément $r \in R$ correspond à un flux et associe à chaque document d son classement pour celui-ci, et k est une constante qui est usuellement choisie égale à 60.

Les deux premiers flux opèrent au niveau global sur toute la base de code : le flux *Dense* cherche les concepts, tandis que le flux *Sparse* (BM25) vérifie que les mots demandés sont présents. Les deux flux suivants sont conditionnels et dédiés au "Source Boosting" : si l'agent spécifie un chemin de fichier, Qdrant relance une recherche Dense et une recherche Sparse uniquement à l'intérieur de ces fichiers cibles. Cette architecture matricielle (cf. Table 1) garantit que peu importe si l'utilisateur pose une question large et conceptuelle ou s'il cherche l'usage exact d'une fonction spécifique dans un dossier précis, le système a mathématiquement la capacité de trouver le bon chunk sans nécessiter de calcul post-recherche.

Le principal défaut de cette architecture RAG est que le RRF est aveugle aux scores réels et ne fusionne que les classements (rangs). Si l'agent LLM hallucine et demande de chercher le mot-clé `forecasting` dans un fichier source qui n'a rien à voir (ex : `/1. utilities/Modules/Colors`), les flux 3 et 4 trouveraient quand même des résultats dans ce fichier, et le rang #1 de ces flux obtiendrait un boost massif, enterrant la vraie réponse. Pour contrer partiellement cela, nous avons équipé Qdrant de seuils de score pour les flux de source boosting et nous les avons ajustés expérimentalement. Ainsi, si la meilleure correspondance dans le fichier cible n'atteint pas un seuil minimal de pertinence (cosine-similarity ou BM25), la recherche ciblée est purement et simplement avortée. Le système RRF retombe alors automatiquement sur les flux globaux (1 et 2), sauvant la

Portée / Méthode	Dense (Sémantique)	Sparse (Lexical / BM25)
Global	**Flux 1 :** Capture les concepts, la logique métier et les intentions globales.	**Flux 2 :** Capture la syntaxe exacte, les noms de variables et les identifiants techniques.
Filtré (Sources)	**Flux 3 (Boosté) :** Recherche conceptuelle limitée aux fichiers cibles. *Seuil : 0.3*	**Flux 4 (Boosté) :** Recherche de mots-clés exacte limitée aux fichiers cibles. *Seuil : 2.0*

Table 1 – Matrice de recherche hybride fusionnée via Reciprocal Rank Fusion (RRF).

recherche et garantissant que le boost de source n'est appliqué que lorsqu'il est factuellement mérité, offrant une meilleure résilience face aux hallucinations de l'agent.

Enfin, nous avons ajouté une ultime couche de raffinement contextuel : le Reranking avec un Cross-Encoder (Jina AI). Qdrant agit comme un filet large et intelligent, remontant les 15 ou 30 meilleurs candidats, mais ces modèles utilisent la similarité cosinus, qui reste une comparaison mathématique relativement simple. Le modèle `jinaai/jina-reranker-v2-base-multilingual` prend le relais en utilisant un *transformer* pour lire simultanément la question de l'utilisateur et le code récupéré. Nous avons choisi Jina spécifiquement pour son excellence dans un environnement bilingue, lui permettant de comprendre parfaitement le lien sémantique entre une question posée en français, des chemins de fichiers français (`/4. Optimization workflow/French/...`), et de la logique métier codée en anglais. Ce modèle ne fait pas que vérifier la présence des mots ; il comprend profondément le contexte technique et évalue si le chunk répond réellement à la question.

Dans le but de maximiser l'efficacité du reranking, nous ne passons pas simplement le code brut au Cross-Encoder, nous concaténons le chemin du fichier au sommet du document (`Source: ...`), et nous structurons la requête en y intégrant explicitement les directives de l'agent (`Required keywords: ... Target source: ...`). Le transformeur Jina peut ainsi lier son attention directement entre le chemin demandé et la métadonnée du chunk. Les scores bruts (logits) générés par Jina sont ensuite normalisés via une fonction Sigmoïde pour obtenir un score normalisé entre 0.0 et 1.0. Cette architecture finale combine la vitesse et la couverture exhaustive de la base Qdrant avec l'analyse chirurgicale et sémantique d'un modèle neuronal lourd, garantissant que seuls les extraits de code les plus pertinents et exacts atterrissent dans la fenêtre de contexte de l'agent final.

3.3.4 • Grep tool : la recherche lexicale exacte guidée par la structure du code

Le `grep_tool` répond à un besoin complémentaire du *RAG*. Lorsqu'une question porte sur un identifiant précis, un chemin exact, un littéral ou une construction syntaxique bien déterminée, une recherche sémantique pure devient trop floue. Dans ce cas, nous préférons un outil de recherche textuelle exacte, piloté par expressions régulières.

Cependant, notre `grep_tool` n'est pas un simple appel à `rg` ou à `grep` sur des fichiers bruts. Il opère sur notre représentation déjà parsée du code Envision : les scripts sont d'abord découpés en blocs syntaxiques cohérents (`IMPORT`, `READ`, `WRITE`, `SHOW`, `EXPORT`, etc.), puis la recherche s'applique sur ces blocs. Cela permet de restreindre la recherche à certaines formes de code, ce qui est particulièrement utile pour répondre à des questions comme *« où lit-on `/Clean/Items.ion` ? »* ou *« dans quels inspecteurs apparaît un `show table` ? »*.

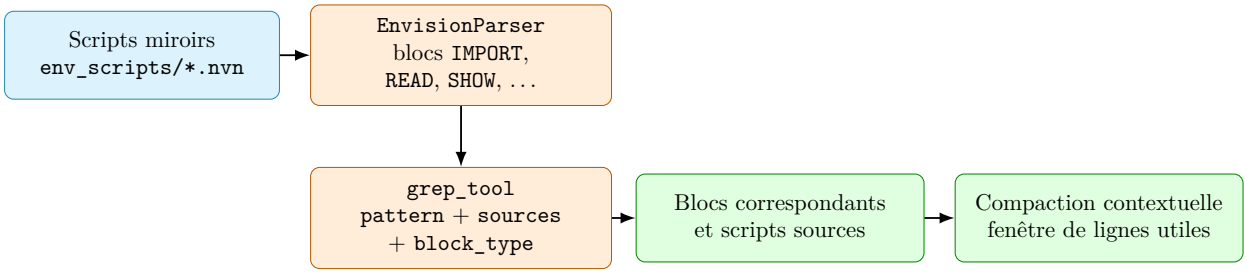

Figure 5 – Chaîne de traitement du `grep_tool` : la recherche ne s'effectue pas sur du texte brut, mais sur des blocs Envision parsés puis compactés avant renvoi au modèle.

Trois leviers de contrôle L'outil est construit autour de trois paramètres simples mais très efficaces :

- `pattern` : le motif recherché, généralement un identifiant, une chaîne ou une petite expression régulière.
- `sources` : un filtre optionnel sur les chemins de scripts, pour restreindre la recherche à une zone du dépôt.
- `block_type` : une liste optionnelle de types de blocs Envision, permettant par exemple de ne chercher que dans les `READ` ou les `SHOW`.

Cette combinaison donne au planificateur un contrôle beaucoup plus fin qu'une recherche globale. Il peut d'abord cibler un dossier logique, puis resserrer encore la recherche au niveau d'une structure syntaxique particulière.

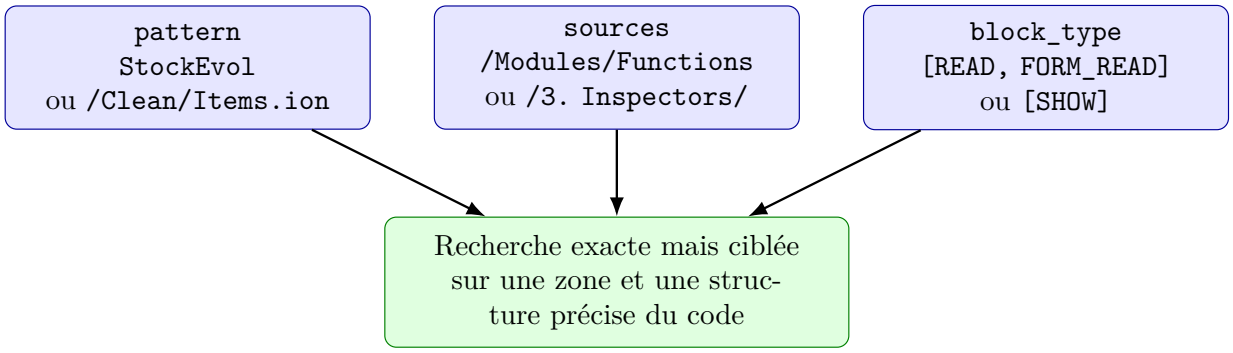

Figure 6 – Les trois degrés de liberté du `grep_tool`. Le planificateur peut jouer sur le motif, la zone du dépôt et le type syntaxique recherché.

Une difficulté majeure que nous avons dû surmonter concerne la définition itérative des noms de fichiers dans Envision. Le code utilise fréquemment des interpolations de chaînes de caractères pour définir les chemins d'importation, par exemple :

```
const testStorage = "/Data/"
const inputPath = "\{testStorage}Clean/"
read "\{inputPath}Items.ion" as Items
```

Une recherche textuelle simple (Grep) sur la chaîne `"/Clean/Items.ion"` échoue ici, car le chemin complet n'est jamais écrit explicitement en une seule ligne.
Nous avons donc dû ajuster l'outil Grep pour gérer cette complexité. L'outil effectue désormais une résolution récursive à la volée : il isole la dernière partie du chemin, vérifie si le préfixe correspond à une variable définie précédemment (comme `{inputPath}`), et remonte ainsi la chaîne de dépendances jusqu'à la racine. Bien que fonctionnelle, cette méthode est coûteuse en temps de calcul car elle s'exécute au moment de la requête (*runtime*).

Exemples réels sur le dépôt courant Les requêtes ci-dessous ont été exécutées sur l'état actuel du dépôt.

— La requête `pattern="StockEvol"` avec `sources="/Modules/Functions"` retourne un unique résultat, le module `/1. utilities/Modules/Functions.nvn`. Ce cas est typique d'une recherche d'identifiant précis.
— La requête `pattern="show table"`, `sources="/3. Inspectors/"` et `block_type=[SHOW]` retourne 16 résultats, tous situés dans les inspecteurs. Ici, le filtre syntaxique évite de rechercher inutilement dans les blocs de calcul.
— La requête `pattern="/Clean/Items.ion"` avec `block_type=[READ, FORM_READ]` retourne 31 résultats. Le `grep_tool` identifie alors précisément les scripts qui lisent ce fichier, avant de compacter les extraits pertinents.

```
read "/Clean/Items.ion" as Items[Id unsafe] with
  "Sku" as Id : text
  Name : text
  Category : text
  SellPrice : number
  BuyPrice : number
  StockOnHand : number
  ...
```

L'extrait ci-dessus illustre le format final transmis au modèle : le bloc est raccourci autour de la zone utile, avec une fenêtre de contexte ajustée automatiquement pour respecter un budget total de lignes.

Apport pour l'agent Le `grep_tool` joue donc le rôle d'un instrument de **précision lexicale**. Là où le *RAG* répond mieux aux formulations conceptuelles et où le `graph_tool` fournit des relations structurelles, le `grep_tool` excelle pour :

— retrouver un identifiant exact ;
— confirmer qu'un chemin ou un littéral apparaît réellement dans le code ;
— restreindre une recherche à un dossier particulier ;
— cibler une forme syntaxique précise, comme les `READ` ou les `SHOW`.

Cette complémentarité est importante dans notre architecture : elle permet au planificateur de choisir entre une recherche **sémantique**, **lexicale** ou **structurelle** selon la nature effective de la question.

3.3.5 • Graph tool : un graphe de dépendances structurelles

Le `graph_tool` complète le *RAG* sémantique et la recherche lexicale par une troisième vue du code : une vue **structurelle**. L'idée n'est plus seulement de retrouver des morceaux de texte pertinents, mais de reconstruire explicitement les dépendances du projet sous la forme d'un graphe navigable.

À la différence du graphe d'orchestration *LangGraph*, qui pilote la boucle de raisonnement de l'agent, le `graph_tool` représente le contenu même du dépôt Envision. Chaque script, chaque dossier logique, chaque fichier de données, ainsi que certaines entités internes (tables, fonctions) deviennent des nœuds ; les relations `imports`, `reads`, `writes`, `defines`, `contains` et `sibling` deviennent des arêtes. Ce graphe permet donc de répondre à des questions structurelles du type : *« quels scripts lisent `/Clean/Items.ion` ? »*, *« quel module est importé par tel inspecteur ? »*, ou encore *« dans quel script une table est-elle définie ? »*.

Deux arbres fondamentaux reliés Le graphe n'est pas un arbre unique. Il juxtapose deux hiérarchies distinctes :

— le domaine **scripts**, qui organise les fichiers `.nvn` selon leur arborescence logique ;
— le domaine **data**, qui organise les fichiers `.ion`, `.csv` ou `.xlsx` manipulés par les scripts.

Ces deux arbres sont ensuite reliés par des arêtes transverses de lecture et d'écriture. Le graphe encode ainsi en une seule structure à la fois la hiérarchie du code et le flot des données.

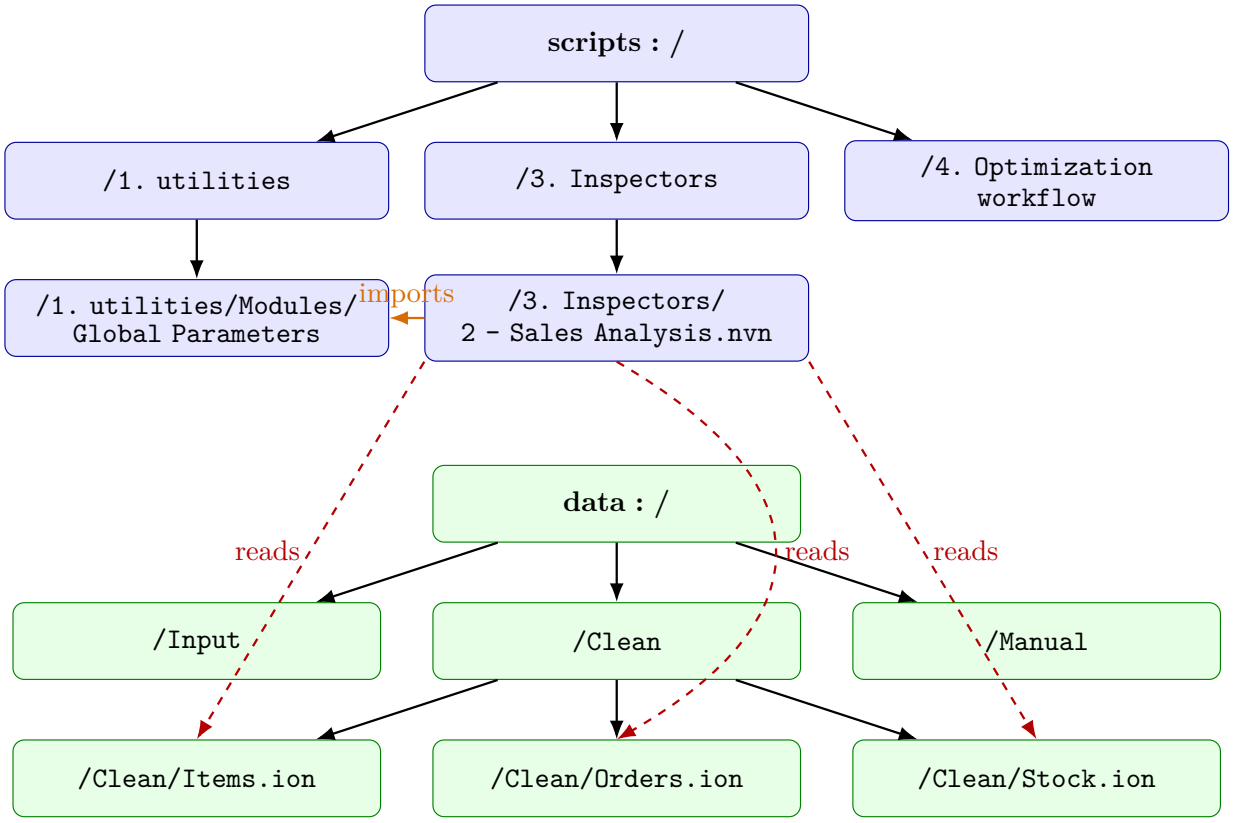

Figure 7 – Visualisation conceptuelle du graphe interne : deux arbres fondamentaux (*scripts* et *data*) reliés par des arêtes de dépendance.

Exemple local de sous-graphe Une fois le graphe construit, l'agent peut raisonner localement autour d'un nœud particulier. La figure suivante montre le sous-graphe centré sur `/3. Inspectors/2 - Sales Analysis.nvn`, qui importe un module, lit plusieurs fichiers de données et définit une table.

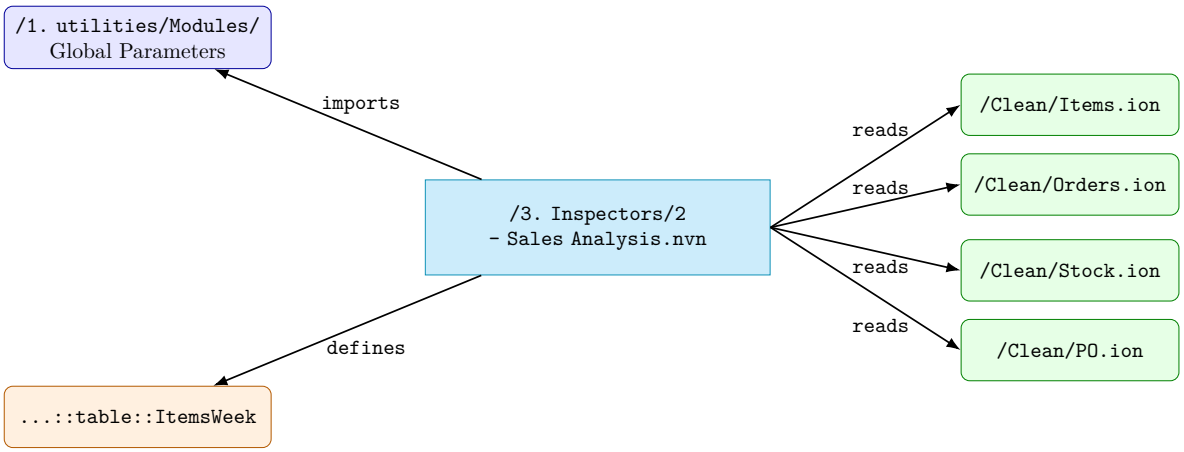

Figure 8 – Sous-graphe typique autour d'un script : le graphe fournit directement les relations structurelles utiles à l'agent.

Statistiques réelles du graphe Sur l'état courant du dépôt, le graphe généré le 11 avril 2026 contient 300 nœuds et 781 arêtes. La répartition est la suivante : 64 scripts, 73 fichiers de données, 96 tables, 17 fonctions et 50 dossiers. Côté relations, on compte notamment 55 arêtes `imports`, 270 arêtes `reads`, 63 arêtes `writes`, 113 arêtes `defines` et 185 arêtes `contains`. Cette densité montre bien que le *graph tool* n'est pas un simple affichage d'arborescence, mais une représentation réellement relationnelle du dépôt.

Usage pratique Le graphe peut être interrogé indépendamment de l'agent via le CLI `python -m env_graph.network`. En pratique, ce CLI expose les mêmes familles d'opérations que l'outil agentique : lecture des statistiques globales, exploration d'un arbre `scripts` ou `data`, recherche d'un nœud par son nom, inspection de ses métadonnées, lecture locale de son contenu et navigation dans ses voisins entrants ou sortants. Il permet donc aussi bien une exploration manuelle du dépôt qu'une validation rapide de ce que l'agent utilise ensuite dans la boucle de raisonnement.

Apport pour l'agent Le `graph_tool` apporte donc une forme de **mémoire structurelle précompilée**. Là où le *RAG* sémantique répond bien aux questions de logique métier et où le `grep` répond bien aux recherches exactes, le graphe fournit une vérité relationnelle sur le dépôt :

- qui lit quoi ;
- qui écrit quoi ;
- qui importe quoi ;
- dans quel dossier logique se trouve chaque élément ;
- quelles tables ou fonctions sont définies par quel script.

Cette couche est particulièrement précieuse dans Envision car de nombreux chemins sont construits dynamiquement par interpolation de chaînes. Le graphe capture et normalise une partie de ces dépendances en amont, ce qui réduit la charge d'inférence précédemment évoquée avec le `grep_tool`.

3.4 Query Transformers

Le problème que l'on peut rencontrer si l'on procède à un simple embedding de la question de l'utilisateur avant *retrieval* dans l'index est l'écart sémantique entre question et documents indexés. Les query transformers cherchent à combler cet écart en retraitant la question par reformulation ou enrichissement.

3.4.1 • RAG Fusion

Le RAG fusion [19] consiste à d'abord demander à un LLM de reformuler la question en plusieurs sous-questions, puis utiliser le RAG pour obtenir les documents les plus adaptés à chacune de ces questions. On classe ensuite les résultats à l'aide de la formule de RRF détaillée plus haut.

En pratique, nous avons constaté que séparer la question en sous-questions fonctionnait moins bien que de proposer plusieurs formulations de la question ayant toutes le même sens. En effet, les réponses aux sous-questions ne peuvent souvent pas être combinées pour répondre à la question initiale alors que les différentes formulations permettent d'élargir le spectre lexical pour la recherche dans les documents. Il est important d'inclure la question d'origine dans la recherche, sans quoi un aspect de la question risque de ne pas se retrouver dans les autres formulations et donc d'être ignoré.

3.4.2 • HyDE

La démarche avec HyDE [20] (Hypothetical Document Embedding) est différente. Dans les approches prédécentes, nous cherchions les documents contenant les réponses via l'embedding des questions posées. Le problème avec ce type d'approche était l'écart sémantique qui pouvait exister entre une question et sa réponse. Pour pallier cela, nous générons des documents hypothétiques censés répondre à notre question. Nous utilisons ensuite l'embedding de ces documents idéaux pour le retrieval dans l'index au lieu de l'embedding d'une ou plusieurs

questions. Un exemple concret : pour la question "quels sont les symptômes de la maladie X ?", il parait intéressant de se concentrer sur les documents qui font intervenir les embeddings de mots comme "fièvre", "fatigue". Sans HyDE, nous ne ciblons pas directement ces mots car nous utilisons l'embedding de la question qui n'inclut que la notion vague de "symptôme".

4
CONTEXT ENGINEERING

Bien que les modèles récents acceptent des contextes de plus en plus larges (jusqu'à 128k tokens), injecter l'intégralité du code source d'un projet dans le prompt est une stratégie inefficace. Non seulement cela augmente drastiquement la latence et les coûts, mais cela dilue l'information pertinente dans du bruit ("Lost in the Middle phenomenon") [21]. Il est donc primordial de gérer correctement le contexte fourni au LLM.

4.1 Les instructions de base

Le système de *context engineering* s'appuie sur un prompt fondateur qui guide tout le raisonnement du LLM. Il définit la philosophie à suivre par le biais des consignes censées orienter le raisonnement du modèle. Par exemple, on indique au LLM de chercher en priorité dans les scripts et non dans les fichiers de données qui sont bien souvent moins pertinents pour répondre à une question sur une mécanique globale.

```
Critical Mental Model
    When a user asks "where is X configured?" or "how do I modify Y?", the safest default is to look
        for the script that WRITES the relevant table or file.
    If a data file is only being READ, it is often downstream consumption; the business logic is
        usually upstream in the script that produced it.
```

Ce document définit également des connaissances essentielles sur la syntaxe Envision pour permettre au LLM de mieux identifier différentes sections du code. Le langage Envision étant un langage interne à Lokad, cette section est absolument essentielle. En effet, le LLM peut procéder par analogie avec d'autres langages plus populaires ou comprendre grâce au contexte mais il demeure qu'il n'a pas été entraîné dessus auparavant et n'en maîtrise donc pas la syntaxe.

```
Variables and Tables
    Vectors vs. Scalars: * Vectors (columns): Identified by dot notation (e.g., Orders.Amount).
    Scalars (single values): Identified without a dot (e.g., exchangeRate).
    Dimensions: Tables typically define a primary key within brackets (e.g., Items[id]).
```

Le document `base_instructions.txt` est unique et utilisé à chaque étape du *planning* afin d'assurer un raisonnement cohérent entre chaque appel.

4.2 Le Distillation Tool

Une fois le retrieval effectué, nous obtenons des `RetrievalResults` qui contiennent des chunks entiers, dont le volume d'information apparait superflu par rapport à la question initiale.La mission du distillation tool est de condenser ces `RetrievalResults` en des faits concis, en rapport avec la question. Dans le cas où un fait est redondant entre les chunks, on n'en garde alors qu'un seul exemplaire.

Notons que nous avons recours à de la *Lazy Distillation* : nous n'effectuons pas la phase de distillation immédiatement après la récupération d'informations par un outil, mais plutôt lors de la phase de planning qui suit. Cela permet d'éviter de condenser des `RetrievalResults` si le planner décide de rendre une réponse définitive à l'utilisateur.

```
_DISTILLATION_SYSTEM_PROMPT = (
    "You are the Memory Manager of a complex RAG Agent. "
    "Your sole responsibility is to extract and preserve key facts "
    "so that the agent can answer its query without re-reading those documents.")
```

Cette distillation a lieu entre les différents appels au *Planner*, avec accès aux résultats bruts et à la tentative précédente du LLM principal afin de compresser au mieux l'information utile.

4.3 Le Prior Evidence Tool

Le `prior_evidence_tool` est un autre élément important du *context engineering*. Il permet au *Planner* de réaccéder à des informations précédentes via un système d'Id sans relancer une recherche par les *tools*. Cela permet d'économiser des recherches inutiles, de combiner des informations issues d'appels différents et d'accéder à des faits issus de questions précédentes lorsque l'utilisateur est en mode conversationnel.

4.4 Knowledge Bank / Accumulated Evidence / Execution History

Nous utilisons trois "banques" de données qui permettent de conserver l'information et le contexte des opérations déjà réalisées. Chacune de ces banques possède une utilité différente, comme détaillé par le tableau suivant.

Nom	**Type de Donnée**	**Objectif**	**Source**
Knowledge Bank	List[Knowledge Element]	Faits distillés donnés en contexte au *Planner*	Incrémenté par le *Distillation Tool*
Accumulated Evidence	Dict[Id → Retrieval Result]	Banque de résultats bruts accessible via le `prior_evidence_tool`	Incrémenté par le *Distillation Tool*
Execution History	List[ActionLog]	Résumé des appels précédents fourni au *Planner* pour éviter la redondance.	Incrémenté directement par les *tools*

Table 2 – Synthèse des structures de données du projet

Ici, il est important de noter que la `Knowledge Bank` ainsi qu'une version condensée de `l'Execution History` sont fournis dans le *planner prompt* avec les mentions *VERIFIED FACTS* et *HISTORY*. En aucun cas le *planner* ne reçoit les données brutes non condensées comme celles que contient `Accumulated Evidence`. Cette dernière banque fait office de base de données consultable au travers du `prior_evidence_tool` par le *Planner* s'il en prend la décision.

4.5 Motivations derrière cette implémentation

Les motivations derrière cette implémentation sont multiples :

- **Efficacité en termes de tokens :** on réduit la taille du contexte en le limitant aux faits pertinents liés à la question. On évite des appels répétitifs grâce à l'historique des appels.
- **Traçabilité du raisonnement :** Ce système permet de suivre le cheminement du raisonnement via le système d'`ActionLogs` et d'*Ids* liés aux faits. Cela permet à l'utilisateur de comprendre le cheminement et au développeur de debugger plus aisément.
- **Mode conversationnel avec mémoire :** Puisqu'on garde la trace des appels précédents, l'utilisateur peut exploiter ses anciennes questions lors d'une conversation avec le LLM.

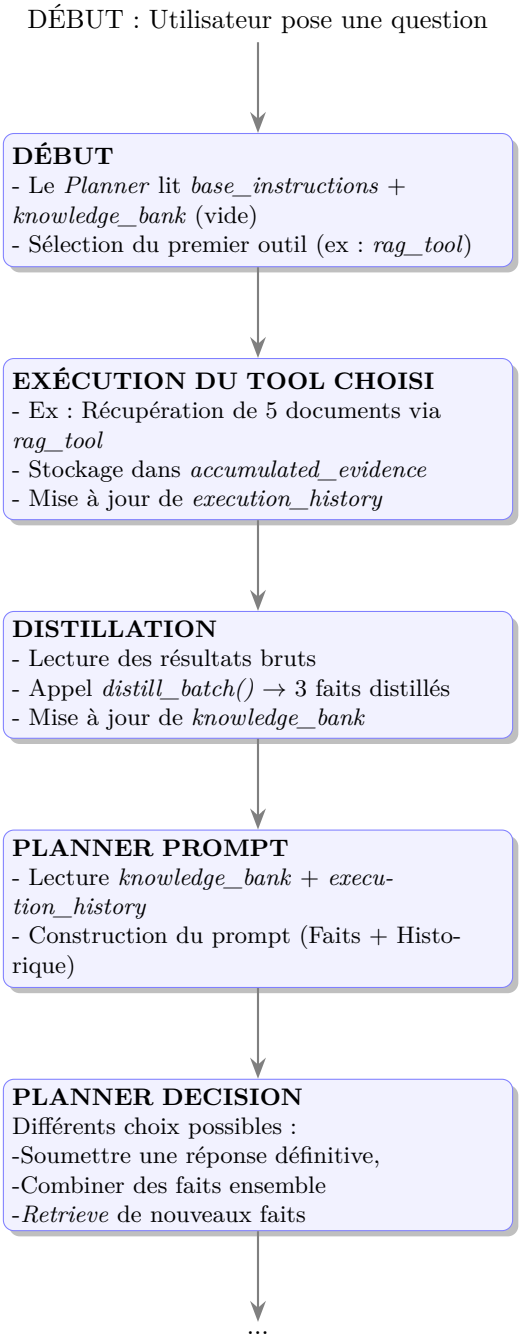

Figure 9 – Exemple illustratif de la gestion du contexte (sans *Lazy Distillation*)

5
PROTOCOLE D'ÉVALUATION (BENCHMARKING)

Le développement d'une architecture agentique complexe ne constitue que la moitié du travail d'ingénierie. L'autre moitié, tout aussi cruciale, réside dans la capacité à mesurer objectivement la performance du système. L'évaluation automatique d'un assistant IA est un domaine notoirement difficile, car contrairement à des tâches de classification, il n'existe pas toujours une unique "bonne réponse" syntaxique pour un problème donné.

5.1 Le défi de la métrique : L'illusion de la similarité

Dans les systèmes de NLP classiques, l'étalon-or est souvent la *Cosine Similarity* (similarité cosinus). Cette métrique géométrique calcule l'angle entre le vecteur de la réponse générée et celui de la réponse de référence (Vérité Terrain). Si l'angle est faible, les textes sont considérés comme sémantiquement proches.

Cependant, dans le contexte rigoureux d'un DSL comme Envision, cette approche s'est révélée imprécise, voire contre-productive : un modèle peut générer une réponse syntaxiquement parfaite, utilisant le bon vocabulaire technique, ce qui lui confère un score de similarité très élevé. Pourtant, une simple négation dans une phrase ou une référence à la mauvaise table (`Orders` au lieu de `OrderLines`) rend la réponse objectivement fausse. La métrique vectorielle ne capture pas cette "rupture" logique.

5.2 La solution polyvalente : "LLM-as-a-Judge"

Pour pallier ces biais structurels, nous avons implémenté une méthode d'évaluation moderne basée sur un **Juge Artificiel**, inspirée des travaux récents sur l'évaluation des chatbots [22].

Le principe est de déléguer l'évaluation à un LLM tiers (dans notre cas, un modèle comme `mistral` ou `deepseek-v3`) dont l'unique tâche est de comparer la réponse de l'agent avec la vérité terrain. Il analyse la cohérence logique et factuelle de la réponse. Contrairement à la solution "cosine similarity", on garantit qu'aucune réponse fausse mais sémantiquement similaire (en terme d'*embedding*) ne soit valorisée et on récompense les réponses justes et appropriées.

5.3 L'utilisation du dual cross-encoder

Comme expliqué précédement dans ce rapport, un *cross-encoder* est un modèle entraîné pour comparer directement deux documents et donner un score de similarité.

Concrètement, le cross-encoder utilise une architecture de transformer. Il concatène les deux documents grâce à un séparateur spécial ce qui permet aux tokens des deux passages de s'influencer entre eux.

L'approche duale combine deux perspectives. Tout d'abord, la cohérence logique (NLI - Natural Language Interference, modèle : DeBERTa v3) : on évalue si la réponse générée est logiquement cohérente avec la réponse attendue ou si des contradictions factuelles existent. En quelque sorte, il s'agit de garantir la justesse factuelle. Ensuite, la pertinence contextuelle (modèle : Jina) : on cherche à mesurer le degré d'alignement entre la question posée et la réponse générée. Il s'agit d'observer la pertinence et l'utilité de la réponse.

Enfin le modèle vérifie d'abord s'il n'y a pas de contradiction majeure, auquel cas, le score sera zéro. Sinon, il combine les deux critères. Ce benchmark a l'avantage de fournir un scoring continu contrairement au *LLM-as-a-Judge* mais est beaucoup plus compliqué à faire fonctionner en pratique du fait de son grand nombre

d'hyper-paramètres à ajuster (Quel est le seuil d'une contradiction majeure ? Comment combiner les 2 critères ? etc).

5.4 La solution hybride

Les deux solutions présentées ci-dessus permettent de mieux noter les réponses. En effet, la réponse proposée, même si elle est proche en termes géométriques, n'obtiendra pas une bonne note si elle est incorrecte. Cependant le problème qui se pose est celui de l'utilisation d'un modèle (LLM ou Cross-Encoder), souvent coûteux, afin de vérifier que la réponse est juste.

Or, lorsqu'il s'agit de retourner un nombre précis ou une liste de fichiers exhaustive pour une question structurelle ou syntaxique, la notation attendue est binaire : "*les éléments attendus sont-ils présents ou non ?*". Nous pouvons donc simplement faire une recherche par regex dans la réponse produite par notre architecture, pour vérifier que la donnée précise y est contenue. Cette approche a de plus l'avantage d'être infaillible sur les questions structurelles contrairement aux approches reposant sur les transformers qui sont plus heuristiques.

Ainsi, nous pouvons facilement et efficacement corriger les questions structurelles, et faire le choix d'une correction par la méthode *LLM-as-a-judge* ou *Dual Cross-Encoder* pour les questions sémantiques.

5.5 Constitution du Benchmark

Pour alimenter le benchmark, nous avons construit manuellement un jeu de données de test composé de paires questions-réponses issues de l'historique réel de développement de Lokad. Ce dataset est segmenté en deux catégories distinctes pour tester les différentes capacités de notre architecture hybride :

1. **Questions structurelles/syntaxiques (Test du GREP) :**
   Exemple : "Quels scripts lisent le fichier `/Clean/Items.ion` ?"
   Attendu : Une liste exhaustive et exacte des noms de fichiers. Ici, l'évaluation est binaire : la liste est complète ou elle ne l'est pas. C'est un test de la capacité de l'agent à naviguer dans la structure du projet.
2. **Questions sémantiques (Test du RAG) :**
   Exemple : "Où est définie la logique d'actualisation du stock ?"
   Attendu : Une réponse pointant vers les bons modules et expliquant brièvement la logique. Ici, le Juge évalue la pertinence des documents retrouvés et la clarté de la synthèse.

5.6 Métriques de performance système

Au-delà de la qualité intrinsèque des réponses (précision), la viabilité industrielle de l'outil dépend de sa performance opérationnelle. Nous avons donc instrumenté notre pipeline pour suivre deux indicateurs critiques :

— **La Latence d'Inférence :** Nous nous intéressons au temps écoulé entre la requête utilisateur et la réponse finale. Actuellement, une boucle agentique complète (réflexion, appel d'outil, génération) prend entre 5 et 20 secondes. C'est acceptable pour un assistant de développement, mais une piste d'amélioration consiste à réduire ce délai en optimisant les appels réseaux.

— **Le Temps de Construction de la base de données :** Une métrique souvent négligée mais vitale. Le code source d'Envision évolue tous les jours. Si notre système met 30 minutes à "lire" et indexer la base de code, il sera toujours obsolète.
Grâce à notre parser optimisé, nous parvenons actuellement à parser, chunker et indexer l'ensemble du corpus (plusieurs centaines de scripts) en moins de **1 minute**.

5.7 Evaluation du benchmark

Pour vérifier la qualité du benchmark, nous avons implémenté un benchmark “pour le benchmark” : nous avons généré un certain nombre de réponses aux questions du benchmark, complété par des versions modifiées de ces réponses pour avoir des exemples de réponses correctes et incorrectes pour chaque question. Pour chacune de ces réponses, nous avons indiqué si elle était correcte ou non, puis nous avons lancé le benchmark et comparé les notes du benchmark avec nos indications. *LLM-as-a-judge* obtient un score entre 19/20 et 20/20 sur ce “benchmark pour benchmark”, confirmant que notre benchmark est plutôt fiable.

6 RÉSULTATS

Comme expliqué précédemment, nous avons exploré plusieurs pistes pour évaluer les performances de notre modèle. L'approche hybride s'est avérée la plus pertinente pour présenter les résultats suivants : elle combine la fiabilité de la vérification par expression régulière pour les questions déterministes à la flexibilité d'un évaluateur LLM (*LLM-as-a-Judge*) pour les questions ouvertes. Notons qu'elle obtient naturellement un score comparable au *LLM-as-a-Judge* dans notre "Benchmark pour Benchmark".

6.1 Comparaison des Modèles de Langage

Nous avons évalué la capacité de l'agent à résoudre un benchmark de 27 questions de difficulté variable portant sur la base de code. Le graphique suivant compare le nombre de réponses correctes obtenues selon la configuration du modèle.

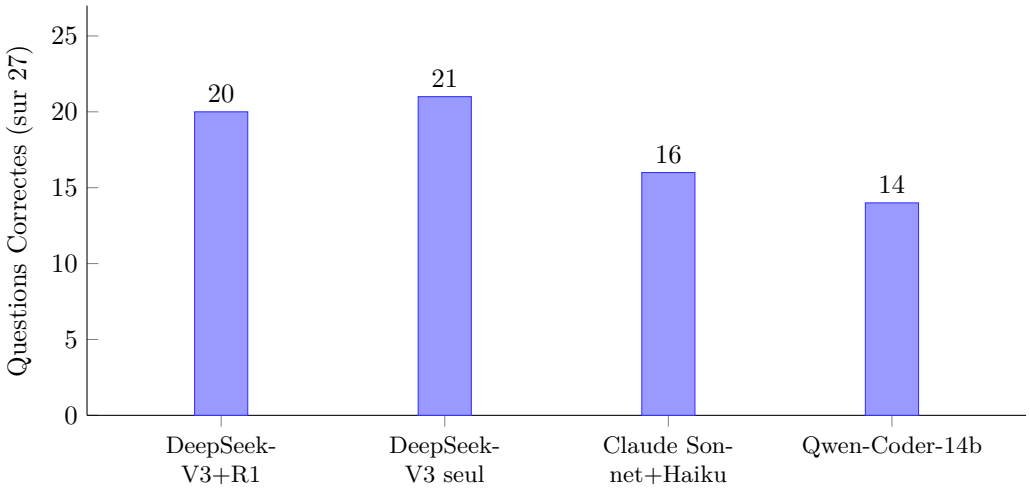

Figure 10 – Performance comparative des LLM sur le benchmark de 27 questions.

Les meilleurs résultats sont clairement obtenus avec DeepSeek-V3. L'utilisation de DeepSeek-R1 comme `Solver LLM` offre des performances très similaires, mais allonge significativement le temps d'inférence (+15 % en moyenne par question). Bien que nos prompts aient été optimisés spécifiquement pour DeepSeek, l'écart de cinq bonnes réponses avec Qwen et Claude confirme la supériorité algorithmique du modèle chinois pour notre cas d'usage.

Ce résultat était prévisible vis-à-vis de `qwen2.5-coder:14b-instruct`, car DeepSeek-V3 possède 685 milliards de paramètres, lui conférant une capacité de raisonnement et une flexibilité bien supérieures. Toutefois, nous nous attendions à des performances comparables, voire meilleures, de la part de Claude, réputé pour son excellence en analyse et génération de code. Une analyse détaillée de ses échecs révèle que Claude a souvent tendance à halluciner des retours d'outils pour inventer des sources censées étayer ses conclusions. Ce comportement s'explique probablement par notre structuration de contexte, qui diffère drastiquement des conventions standards attendues par les modèles Anthropic. Claude étant un modèle propriétaire, il est très sensible aux écarts de format par rapport à ses données d'entraînement. À l'inverse, les modèles DeepSeek se montrent beaucoup plus agnostiques et résilients face à des formats de prompts atypiques.

Enfin, sur le plan économique, l'exécution complète du benchmark coûte environ 0,15 € avec DeepSeek, contre près de 4 € avec Claude. Étant à la fois plus performant et nettement moins onéreux, DeepSeek-V3 a donc été retenu comme modèle de référence pour construire tous les graphiques suivants.

6.2 Analyse de l'Utilisation des Outils et de la Latence

L'agent dispose d'une panoplie d'outils pour explorer le code. La Figure 11 présente la fréquence d'appel à chaque outil durant la résolution du benchmark, tandis que la Figure 12 détaille la répartition du temps d'exécution.

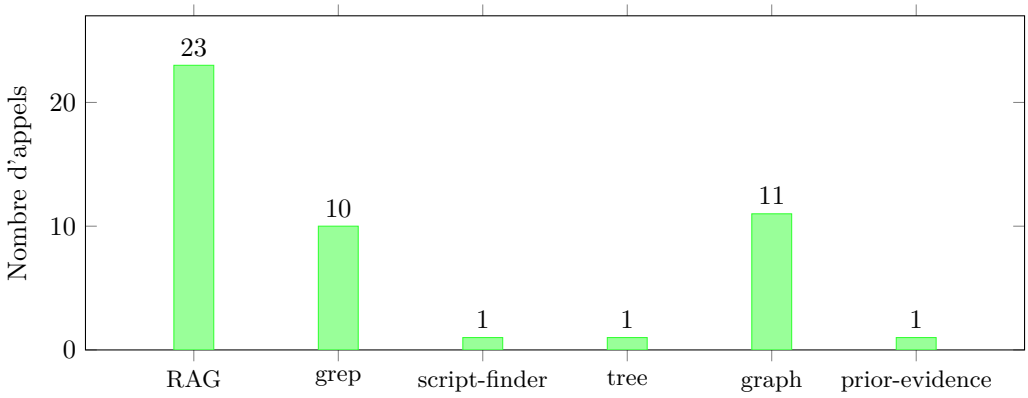

Figure 11 – Fréquence d'utilisation des outils par l'agent.

Pour les 7 premières questions du benchmark, fortement axées sur la syntaxe, l'agent utilise quasi exclusivement le `graph_tool`. Cet outil étant parfaitement adapté à ce type de recherche, il garantit un taux de réussite systématique de 100 % sur cette catégorie. Cela explique la concentration des 11 appels sur cet outil, qui n'est ensuite plus sollicité pour le reste de l'évaluation.

Pour les 20 questions restantes, de nature conceptuelles, le RAG devient l'outil de prédilection en raison de sa polyvalence, encore renforcée par nos mécanismes de *boosting* par mots-clés et par sources. Lorsqu'une variable ou un nom de fonction est explicitement mentionné, l'agent effectue souvent une recherche complémentaire via `grep` pour garantir l'exactitude technique de sa réponse. Enfin, le recours aux autres outils est moins systématique et dépend fortement du cheminement de réflexion adopté par le LLM.

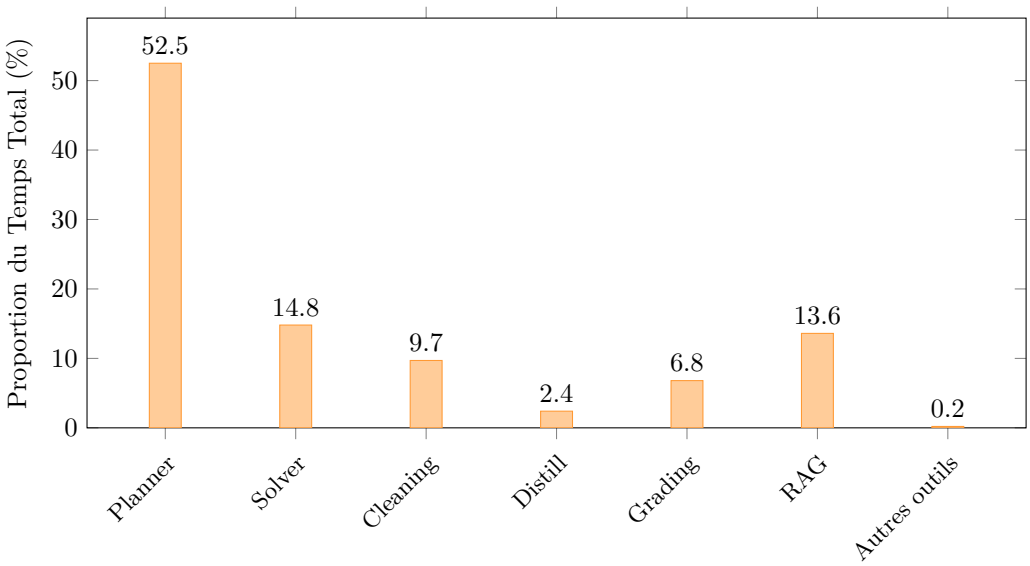

Figure 12 – Répartition du temps d'exécution par composant et rôle LLM pour une durée totale de 30m 12s.

L'analyse de la latence avec DeepSeek-V3 révèle que la majorité du temps d'exécution est monopolisée par le `Planner LLM`. En tant que centre de commande, ce modèle est chargé de sélectionner les outils à appeler pour explorer la base de code. En décomposant ses 52,5 % de temps d'exécution, on constate que 27,2 % sont spécifiquement dédiés à l'évaluation des propositions intermédiaires du `Solver LLM` (décider si la réponse finale est prête ou si de nouvelles recherches sont requises). Cette étape de vérification est volontairement imposée au système : elle force le LLM à mener une analyse critique avant de proposer une nouvelle action, évitant ainsi des appels d'outils précipités et non pertinents.

Le `Solver LLM`, chargé de formuler la réponse finale à l'utilisateur, occupe également une portion significative du temps. Cette proportion peut même atteindre 30 % lorsque l'on utilise DeepSeek-R1, un modèle de raisonnement qui intègre une longue chaîne de pensée (*Chain-of-Thought*) générée directement via l'API.

Enfin, le module RAG a un impact notable sur la latence globale de l'agent (13,6 %). Comme l'illustre la section suivante, ce délai est presque exclusivement imputable à l'étape de *reranking*, qui sollicite un modèle Transformer lourd pour croiser sémantiquement la requête avec les 25 documents candidats à chaque appel du `rag_tool`.

6.3 Évaluation des Améliorations du RAG

L'architecture du RAG a évolué pour pallier les limites de la recherche dense simple. Ce graphique présente l'impact de chaque amélioration sur le score de précision final et le temps total passé dans l'outil de RAG pour un benchmark de 20 questions avec DeepSeek-V3 (nous n'avons gardé que les 20 questions conceptuelles sur les 27 questions totales du benchmark car les questions structurelles ne mobilisent pas le RAG).

Configuration	Score (sur 20)	Δ Score	Temps (s)	Δ Temps (s)
FAISS	8	-6	0.47	-246.70
Qdrant-Hybride	9	-5	2.65	-244.52
Qdrant+Jina (Réf.)	**14**	–	**247.17**	–
No-Summary	6	-8	243.05	-4.12
Fusion	12	-2	519.39	+272.22
HyDE	12	-2	709.18	+462.01

Table 3 – Étude d'ablation : évaluation de l'impact des différentes stratégies par rapport à l'architecture de référence.

Afin d'isoler et de quantifier l'impact de chaque amélioration apportée à notre pipeline de recherche documentaire, nous avons mené une étude d'ablation. En apprentissage automatique, une étude d'ablation consiste à retirer, modifier ou ajouter unitairement des composants d'une architecture complexe afin d'évaluer leur contribution réelle à la performance globale. Le tableau ci-dessus (Table 3) compare six configurations distinctes. La configuration *FAISS* représente notre ligne de base initiale (recherche dense pure). *Qdrant-Hybride* y ajoute la recherche lexicale (BM25) et la fusion RRF. *Qdrant+Jina* introduit l'étape de reranking par Cross-Encoder sur les documents récupérés. L'expérience *No-Summary* reprend cette architecture optimale, mais retire l'utilisation des résumés de code pour l'étape d'embedding, indexant le code brut à la place. Enfin, *Fusion* et *HyDE* testent des stratégies avancées d'augmentation de requêtes par LLM (génération de requêtes multiples pour la première, et génération de document hypothétique pour la seconde).

L'analyse des quatre premiers résultats met en évidence de manière univoque la supériorité de l'architecture combinant Qdrant, le reranking neuronal et l'utilisation de résumés. Le passage de FAISS (8 questions justes sur 20) à Qdrant-Hybride (9/20) montre une légère amélioration grâce au flux sémantique et lexical. Cependant, c'est l'ajout du Cross-Encoder Jina qui crée une véritable rupture technologique, faisant bondir la précision à 14 bonnes réponses sur 20. Ce bond en avant souligne la nécessité d'une analyse profonde du contexte par un Transformer après le premier filtrage vectoriel. Plus important encore, l'expérience *No-Summary* prouve que cette architecture repose fondamentalement sur la qualité de l'indexation initiale : sans les résumés générés pour guider l'embedding (en passant le code brut au modèle d'encodage), le score s'effondre drastiquement à 6/20. L'architecture *Qdrant+Jina* (avec résumés) s'impose donc comme le cœur le plus performant du système, bien qu'il induise naturellement un temps de latence supérieur (environ 247 secondes cumulées pour la résolution des requêtes liées aux 20 questions) lié au coût calculatoire du reranker.

L'évaluation des stratégies d'augmentation de requêtes (*Fusion* et *HyDE*) révèle quant à elle un compromis complexe. Ces deux méthodes atteignent des performances comparables, bien que légèrement inférieures à l'architecture optimale (12 bonnes réponses contre 14). Ce plafonnement s'accompagne surtout d'une explosion significative du temps d'inférence, le processus de génération de LLM supplémentaire propulsant la latence à plus de 519 secondes pour le RAG-Fusion, et au-delà des 709 secondes pour HyDE. Bien que ces méthodes s'avèrent trop coûteuses en temps pour un gain de précision net inexistant, elles présentent un comportement intéressant : les résultats de ces benchmarks ont montré un écart-type plus faible entre les différentes exécutions. En d'autres termes, si RAG-Fusion et HyDE abaissent légèrement le plafond de performance maximale de notre agent, elles semblent le rendre globalement plus prévisible et moins sujet aux variations aléatoires liées à la température des LLM.

6.4 Analyse détaillée des questions du Benchmark

Le tableau ci-dessous répertorie les 27 questions du benchmark, catégorisées selon leur nature. Un code couleur indique le niveau de performance de l'agent pour chacune d'elles.

Table 4 – Liste des 27 questions traitées par l'agent RAG.

N°	Question posée au système
	Questions plutôt Syntaxiques (Recherche de métadonnées et de chemins)
1	Combien de scripts lisent /Clean/Items.ion ?
2	Quels scripts lisent /Clean/Items.ion ?
3	Combien de scripts lisent /Clean/Forecast/Periodic/FcItems.ion ?
4	Quels scripts lisent /Clean/Forecast/Periodic/FcItems.ion ?
5	Quels scripts écrivent /Clean/Forecast/Periodic/FcItems.ion ?
6	Combien de scripts font mention de /Clean/Forecast/Periodic/FcItems.ion ?
7	Quels scripts font mention de /Clean/Forecast/Periodic/FcItems.ion ?
	Questions plutôt Conceptuelles (Compréhension de la logique métier)
8	Quels sont les modules utilisés ?
9	Où sont calculés les meilleurs vendeurs ?
10	Que fait-on dans ce compte ?
11	Le client veut ajouter une nouvelle localisation pour le dispatch. Où paramétrer le champ ReDispatchCycle propre à cette localisation ?
12	Existe-t-il une fonction pour calculer l'évolution du stock dans le temps en tenant compte des arrivages et de la demande ?
13	Je vais recevoir une nouvelle colonne FuturSellPrice dans ma table catalogue. Dans quel script effectuer des checks de cohérence sur les valeurs observées ?
14	Combien de profils de saisonnalité sont appris pour créer le moteur de prévision ?
15	Existe-t-il un endroit où retrouver des informations condensées pour analyser les différents fournisseurs ?
16	Où paramétrer des changements globaux sur les prix de vente afin d'observer leur impact sur la prévision des ventes ?
17	Quelle fenêtre temporelle est prise en compte pour l'apprentissage de la prévision des ventes ?
18	Le produit CT-110-DC est-il traité de façon spécifique dans les scripts ?
19	Comment sont définies les cibles de fill rate à respecter selon les produits du catalogue ?
20	Comment et où est définie la croissance annuelle pour chaque produit ?
21	Comment définit-on le fait qu'une ref soit une top ref ?
22	Est-ce que je peux prendre en compte des ventes retour (quantités négatives) dans mes calculs de KPIs ?
23	Est-ce qu'on pondère l'apprentissage des ventes selon leurs dates pour déterminer une prévision future des ventes ?

Suite sur la page suivante...

Table 4 – Suite de la page précédente

N°	Question posée au système
24	Comment modélise-t-on le coût de posséder une unité d'un produit en stock ?
25	Est-ce que le client a l'autonomie de paramétrer lui-même la cible de taux de satisfaction de la demande (fill rate) ?
26	Le client aimerait suivre l'évolution des ruptures de stock : est-ce qu'un dashboard existe déjà qui pourrait être utile ?
27	Quel type de demande moyenne affiche-t-on au client sur le compte ?

Légende du Benchmark :

Questions réussies plus de 95% du temps — Questions parfois réussies et parfois non — Questions toujours échouées

On constate que les performances de notre agent sont soumises à une part d'aléa : le score moyen se situe autour de 19 bonnes réponses, mais pourrait théoriquement atteindre 24 sur des exécutions extrêmement favorables. Cette variance s'explique par l'utilisation de températures non nulles lors des requêtes aux modèles de langage. Celle-ci est fixée à 0.05 pour le `Planner LLM`, qui doit suivre une logique de décision implacable, mais s'élève à 0.3 pour le `Solver LLM` et le `Distillation LLM`. Cette marge permet de stimuler la créativité textuelle et d'éviter que les modèles ne restent trop prisonniers des formulations exactes de leur corpus d'entraînement.

Pour comprendre concrètement comment notre système fonctionne, voici un résumé de l'exécution de notre agent sur trois questions aux cheminements de résolution distincts :

— **Question 4 : Quels scripts lisent /Clean/Forecast/Periodic/FcItems.ion ?**
Pour cette question technique, le planificateur (*Planner*) identifie une problématique de dépendance structurelle. Au lieu d'utiliser la recherche sémantique ou textuelle qui pourrait générer du bruit, il opte stratégiquement pour le `graph_tool`. L'exécution se déroule en deux temps : un premier appel (`action: "search"`) localise précisément le nœud correspondant au fichier de données dans le graphe. L'agent enchaîne ensuite avec une requête de voisinage (`action: "neighbors"`) en filtrant les relations entrantes de type lecture (`direction: "incoming"`, `relation_type: "reads"`). Le graphe retourne alors les trois scripts exacts figurant dans la référence. Constatant que les preuves structurelles sont irréfutables et complètes, l'agent valide la réponse immédiatement sans exploration supplémentaire, garantissant une précision maximale et un score parfait.

— **Question 18 : Le produit CT-110-DC est-il traité de façon spécifique dans les scripts ?**
Pour cette requête conceptuelle liée à la logique métier (traitement d'exception sur un SKU particulier), le planificateur rejette l'idée d'une simple recherche d'identifiant et privilégie l'outil sémantique hybride (`rag_tool`). Il orchestre sa requête en combinant l'identifiant exact (`CT-110-DC`) avec des mots-clés d'intention (`"product", "specific", "treatment"`). Le RAG remonte 5 fragments de code pertinents. Lors de la phase de génération, le LLM analyse ces *chunks* et identifie une règle d'exclusion *hardcodée* dans le script `Forecast Autodiff.nvn` (lignes 155-157). L'agent rédige alors une réponse affirmative, justifiée par l'extrait de code exact qui censure une période de ventes spécifiques (juin-août 2020) pour ce produit. Le module de vérification (*Logic Check*) valide la solidité des preuves, et le module de nettoyage condense l'explication technique en une réponse fluide, obtenant là encore un score hybride de 1 au benchmark.

— **Question 23 : Est-ce qu'on pondère l'apprentissage des ventes selon leurs dates pour déterminer une prévision future des ventes ?**
Ce troisième cas illustre une limite du système (échec sanctionné par un score de 0). Face à cette question conceptuelle, le planificateur a correctement choisi l'outil `rag_tool` en générant des mots-clés pertinents bilingues (`"pondération", "weight", "date", "apprentissage"`). Le problème survient lors de la récupération (RAG) : le système remonte un chunk mathématiquement exact mais sémantiquement hors-sujet issu du fichier `Final Forecast Plan.nvn`. Ce fragment détaille la pondération *journalière*

pour répartir une prévision déjà calculée, au lieu de la pondération *historique* de l'apprentissage (qui se trouvait dans `Forecast Autodiff`). Piégé par la forte correspondance lexicale (présence des mots "pondération", "date" et "forecast"), l'agent principal construit un raisonnement détaillé, factuel et parfaitement sourcé... mais qui répond à la mauvaise question. Cet exemple souligne la vulnérabilité du RAG face au vocabulaire polysémique ("pondération" signifiant ici deux concepts algorithmiques très distincts dans Envision) et à la formulation de la question.

6.5 Pistes d'amélioration

Grâce aux échanges avec Clément Liu et les *Supply Chain Scientists*, nous avons réussi à faire ressortir quelques pistes d'amélioration.

Tout d'abord, parmi les outils, nous avons observé qu'il pourrait être bénéfique de rajouter dans le `graph_tool` une liaison entre les scripts et les colonnes. Cet ajout permettrait à l'outil d'être plus polyvalent et ainsi d'être plus utilisé qu'il ne l'est pour le moment.

Aussi, pour améliorer l'utilisabilité des outils, car leur nombre tend à augmenter, il peut être utile de prévoir des descriptions et des paramètres consultables pour limiter le contexte initial et donc mieux guider le *planner*. Dans le contexte initial, une description rapide de l'outil pourrait être faite, tandis que le *planner* pourrait demander des précisions sur un outil avant de l'utiliser, incluant les paramètres et des exemples d'utilisation.

Enfin, pour renforcer la fiabilité des réponses, il serait intéressant d'ajouter une étape supplémentaire de vérification à la fin. Nous pourrions par exemple exiger des sources et rejeter des réponses qui n'ont pas de preuves explicites.

Nous ne pouvons pas oublier le fait que notre solution est très sensible à une modification des prompts des agents. En outre, notre benchmark a été construit et testé de nombreuses fois sur un seul compte client. En un sens, il se peut que la solution proposée risque de n'être spécifique qu'à cette base de données, ou du moins, d'être moins efficace sur une autre base de données (overfitting). Concernant le benchmarking lui même, le *LLM-as-a-Judge* qui est utilisé pour une bonne partie de la notation du benchmark, peut être soumis à un biais d'évaluation car nous l'avons nous-même construit.

Conscients que nous étions soumis à de tels risques, nous nous sommes tout de même employés à ne pas tomber dans le piège trop facilement en gardant un esprit critique tout au long du projet.

7 GESTION DE PROJET

7.1 Déroulement et organisation du projet

Pour nous coordonner sur l'avancement du PSC nous nous sommes réunis tous les mercredis après-midi durant les créneaux réservés où nous pouvions alors échanger avec Clément Liu. Il était notre point de contact auprès de Lokad et nous prodiguait une analyse critique, des pistes intéressantes ainsi que les ressources nécessaires à la compréhension des besoins de Lokad. La réunion de mi-parcours avec Joannes Vermorel, CEO de Lokad ainsi que Johannes Lutzeyer, notre référent PSC au département d'informatique nous a permis de redéfinir nos objectifs et d'avoir un premier regard extérieur au projet. En plus de ces créneaux, ce projet a également été l'occasion pour nous de nous documenter sur notre temps libre et ensuite implémenter les outils que nous avions conceptualisés durant nos réunions. Grâce à Git, il était assez aisé pour chacun de travailler lorsqu'il le voulait durant la semaine. Nous pouvions ensuite faire le point le mercredi après midi pour échanger sur l'avancement de nos tâches et fixer de nouveaux objectifs pour la semaine d'après.

Le tableau suivant permet de situer l'évolution du PSC tout au long de l'année :

Tâche	**Sept**	**Oct**	**Nov**	**Déc**	**Janv**	**Févr**	**Mars**	**Avril**
Implémentation outils de base	X	X						
Ajout du Grep		X						
Ajout de LangGraph			X	X				
RAG Fusion				X				
Pipeline agentique					X			
MàJ chunking parsing					X			
Chunk summarizer					X	X		
Reranking						X		
Ajout du Graph tool						X	X	
Hyde							X	
Raptor							X	
Benchmarks								X

Table 5 – Tableau de progression chronologique du PSC

Nous avons débuté le projet par la conception de notre pipeline de RAG, inspirée de celle du PSC réalisé l'année dernière pour Lokad. Cette vision reposait sur une approche classique et linéaire, telle qu'elle est souvent présentée dans la littérature académique sur le RAG.

Nous nous sommes documentés sur les outils de base qui nous paraissaient les plus adaptés, notamment *FAISS*, *sentence transformers*. En novembre, nous avions déjà une base de RAG avec un mode verbose et un type de benchmark afin d'observer directement les apports de chaque ajout à venir.

Néanmoins, la confrontation avec les cas d'usage réels a rapidement invalidé cette approche simpliste. La réalité du terrain nous a poussés vers une architecture Agentique non-linéaire.

Une fois celle-ci implémentée, nous avons pu ajouter brique par brique les outils nécessaires à l'amélioration

du modèle de manière indépendante. Les efforts sont alors concentrés sur l'optimisation de la performance et la fiabilité des réponses. Il fallait déterminer les configurations les plus efficaces pour chaque outil. Ce perfectionnement pouvait aller de la modification des prompts à l'ajustement de variables comme la taille des chunks lors de la création de l'index.

La répartition des tâches durant tout ce processus s'est effectuée naturellement en fonction des goûts et des capacités de chacun.

7.2 Livrables et diffusion

En complément du manuscrit, nous avons préparé les deux supports publics associés au projet : un dépôt technique, qui regroupe l'implémentation et l'historique de développement, et une page publique plus synthétique, destinée à présenter le sujet, la composition du groupe et l'architecture générale du système.

— **Dépôt GitHub du projet** [`https://github.com/ClementLokad/llm-DSL-info-extraction`]
— **Page publique de présentation** [`https://kpihx.github.io/envision-copilot-presentation/`]

Cette séparation nous a permis de distinguer clairement le support de développement complet et le support de communication synthétique demandé pour la restitution finale du PSC.

7.3 Enseignements tirés

Ce projet scientifique collectif nous a appris de nombreuses choses. Tout d'abord, la partie organisationnelle est bien évidemment au coeur du projet. Avec une équipe plus nombreuse que ce à quoi nous étions habitués dans un cadre scolaire, la bonne coordination n'était pas assurée de prime abord.

Le fait d'appartenir à la même section rendait la communication plus aisée puisque nous nous croisions régulièrement en dehors des réunions. Cela permettait d'échanger régulièrement sur l'avancée du projet et débattre sur les implémentations en cours, poser des questions sur les outils des autres etc.

Le thème du projet a été un des moteurs qui ont conduit à la création du groupe. Nous étions effectivement convaincus que cette problématique particulièrement actuelle nous bénéficierait dans notre avenir professionnel. Nous avons eu l'occasion de découvrir plus en profondeur le fonctionnement des LLMs et l'écosystème en mutation permanente dans lequel ils s'inscrivent. Nous avons également pu renforcer d'autres compétences, maîtrise de Python, de Git. La complexité de la tâche nous a aussi permis de développer notre persévérance, notre rigueur et notre curiosité scientifique.

En fin de compte, ce travail de longue haleine nous a beaucoup enseigné que ce soit sur le plan social ou scientifique.

7.4 Conclusion

Nous sommes positivement surpris par les résultats finaux que nous avons obtenus et apprécions l'intérêt concret que présente le produit à l'état final. Nous espérons que Lokad partage cette satisfaction et que notre outil trouvera une véritable application dans le quotidien des *Supply Chain Scientists* auxquels l'outil était destiné. Cette satisfaction que nous éprouvons ne signifie pas que l'ensemble de notre parcours fut parfait. Concernant l'organisation, nous ne disposions pas tous du même socle en compétences informatiques. Cela a parfois compliqué la compréhension du code des autres membres, notamment lors de *pushs* abondants. Nous aurions pu éviter ce problème en utilisant plus tôt des outils comme Copilot pour générer de la documentation et commenter le code de manière efficace. L'indépendance des outils a fortement contribué à simplifier les tâches de chacun mais pouvait parfois inciter à ne se focaliser que sur une partie du projet sans bien en connaître le reste.

RÉFÉRENCES

[1] LOKAD, 2025. *Documentation officielle d'Envision, le DSL de la Supply Chain*. Documentation technique. [En ligne]. Disponible sur : https://docs.lokad.com/ [Consulté le 17/09/2025].

[2] VASWANI, A., et al., 2017. *Attention Is All You Need*. Advances in Neural Information Processing Systems (NeurIPS).

[3] TAUMAN KALAI, A., et al., 2025. *Why Language Models Hallucinate*. arXiv preprint. [En ligne]. Disponible sur : https://arxiv.org/abs/2509.04664

[4] WHITE, J., et al., 2023. *A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT*. Vanderbilt University.

[5] RAJASEKARAN, P., et al., 2025. *Effective Context Engineering for AI Agents*. Note d'ingénierie — retour d'expérience industriel (Anthropic AI). [En ligne]. Disponible sur : https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

[6] GAO, Y., et al., 2023. *Retrieval-Augmented Generation for Large Language Models : A Survey*. arXiv preprint. [En ligne]. Disponible sur : https://arxiv.org/abs/2312.10997

[7] REIMERS, N., GUREVYCH, I., 2019. *Sentence-BERT : Sentence Embeddings using Siamese BERT-Networks*. EMNLP-IJCNLP 2019. [En ligne]. Disponible sur : https://arxiv.org/abs/1908.10084

[8] GAO, L., et al., 2022. *Precise Zero-Shot Dense Retrieval without Relevance Labels*. ACL 2023. [En ligne]. Disponible sur : https://arxiv.org/abs/2212.10496

[9] SARTHI, P., et al., 2024. *RAPTOR : Recursive Abstractive Processing for Tree-Organized Retrieval*. ICLR 2024. [En ligne]. Disponible sur : https://arxiv.org/abs/2401.18059

[10] NOGUEIRA, R., CHO, K., 2019. *Passage Re-ranking with BERT*. arXiv preprint. [En ligne]. Disponible sur : https://arxiv.org/abs/1901.04085

[11] RICE, I., 2025. *How I won the Enterprise RAG Challenge*. Blog technique — retour d'expérience industriel. [En ligne]. Disponible sur : https://abdullin.com/ilya/how-to-build-best-rag/

[12] PRATAP, S., et al., 2025. *The fine art of fine-tuning : A structured review of advanced LLM fine-tuning techniques*. ScienceDirect.

[13] MÜLDER, A., 2023. *Large Language Models for Domain-Specific Language Generation : How to Train Your Dragon*. Article de vulgarisation technique (Medium). [En ligne]. Disponible sur : https://medium.com/itemis/large-language-models-for-domain-specific-language-generation-how-to-train-your-dragon

[14] LangChain AI, 2024. *LangGraph : Building stateful, multi-actor applications with LLMs*. Documentation technique. [En ligne]. Disponible sur : https://langchain-ai.github.io/langgraph/

[15] SINGH, A., et al., 2025. *Agentic Retrieval-Augmented Generation : A Survey on Agentic RAG*. arXiv preprint. [En ligne]. Disponible sur : https://arxiv.org/abs/2501.09136

[16] LIANG, J., et al., 2025. *Reasoning RAG via System 1 or System 2 : A Survey on Reasoning Agentic Retrieval-Augmented Generation for Industry Challenges*. Findings of IJCNLP 2025. [En ligne]. Disponible sur : https://aclanthology.org/2025.findings-ijcnlp.122/

[17] ROBERTSON, S., ZARAGOZA, H., 2009. *The Probabilistic Relevance Framework : BM25 and Beyond*. Foundations and Trends in Information Retrieval. [En ligne]. Disponible sur : https://doi.org/10.1561/1500000019

[18] CORMACK, G. V., CLARKE, C. L. A., BÜTTCHER, S., 2009. *Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods*. SIGIR 2009. [En ligne]. Disponible sur : https://dl.acm.org/doi/10.1145/1571941.1572114

[19] RACKAUCKAS, Z., 2024. *RAG-Fusion : a New Take on Retrieval-Augmented Generation*. arXiv preprint. [En ligne]. Disponible sur : https://arxiv.org/abs/2402.03367

[20] JOSTMANN, M. et WINKELMANN, H., 2024. *Evaluation of Hypothetical Document and Query Embeddings for Information Retrieval Enhancements in the Context of Diverse User Queries*. Wirtschaftsinformatik 2024 Proceedings. [En ligne]. Disponible sur : https://aisel.aisnet.org/wi2024/115

[21] LIU, N. F., et al., 2023. *Lost in the Middle : How Language Models Use Long Contexts*. Transactions of the Association for Computational Linguistics. [En ligne]. Disponible sur : https://arxiv.org/abs/2307.03172

[22] ZHENG, L., et al., 2023. *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. NeurIPS 2023. [En ligne]. Disponible sur : https://arxiv.org/abs/2306.05685